

SafarBot



By

Eshan Elahi

22-ARID-3157

Hammad Zahoor

22-ARID-905

Ahan

22-ARID-883

Supervisor

Dr. Farkhanda Qamar

University Institute of Information Technology

PMAS-Arid Agriculture University Rawalpindi

Pakistan-2026

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software documentation and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Eshan Elahi

Hammad Zahoor

Ahan

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS SE “**SafarBot**” was developed by “**Eshan Elahi, 22-ARID-3157**”, “**Hammad Zahoor, 22-ARID-905**” and “**Ahan, 22-ARID-883**” under the supervision of “**Dr. Farkhanda Qamar**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.

(Dr. Farkhanda Qamar)

Supervisor

(Dr.Saif-Ur-Rehman)

Examiner-I

(Dr.Muhammad Habib)

Examiner-II

Director UIIT

Executive Summary

Online ticket booking systems are widely used today, but many people in Pakistan face challenges using them. Most existing systems are text-based, English-only, and complex, making them difficult for uneducated, elderly, or less tech-savvy users. Additionally, these systems often lack essential features such as fraud detection, real-time travel updates, and instant customer support.

To address these issues, SafarBot has been developed as an AI-powered ticket booking system that is simple, secure, and user-friendly. SafarBot allows users to book tickets via voice or text commands in both Urdu and English, making the platform accessible to a broader audience.

The system incorporates AI-based fraud detection to prevent fake or suspicious bookings and provides real-time travel information, including traffic, weather, and delay updates. A Stripe test mode demonstrates secure digital transactions.

SafarBot is a web-based application built on the MERN stack, with additional AI services implemented in Python/FastAPI with Hugging Face model deployment. It features separate dashboards for users, transport providers, and administrators, enabling efficient management of bookings, routes, and system data.

In summary, SafarBot enhances the online ticket booking experience by combining simplicity, security, and intelligent features, making digital travel services accessible and convenient for everyone.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task. We are greatly indebted to our project supervisor “Dr. Farkhanda Qamar” for personal supervision, advice, valuable guidance and completion of this project. We are deeply indebted to her for encouragement and continual help during this work. We are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Eshan Elahi

Hammad Zahoor

Ahan

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheet
DB	Database
JSON	Javascript Object Notation
HTTPS	Hypertext Transfer Protocol Secure
HTML	HyperText Markup Language
ML	Machine Learning
FR	Functional Requirements
NFR	Non-Functional Requirement
UML	Unified Modeling Language

Table of Contents

CHAPTER 1: INTRODUCTION.....	1
1.1. Brief.....	1
1.1.1. Overview of the Project.....	1
1.1.2. Project Outcomes	1
1.1.3. Tools and Technologies Used.....	2
1.1.4. Methodology Overview.....	2
1.2. Relevance to Course Modules	2
1.2.1 Application of Academic Knowledge.....	2
1.2.2 Course Integration	3
1.2.3 Professional Skill Development.....	3
1.3. Project Background	4
1.3.1 Motivation	4
1.3.2 Problem Context.....	4
1.3.3 Project Vision	4
1.4. Literature Review	4
1.4.1 Existing Systems and Research	5
1.4.2 Current Trends	5
1.4.3 Identified Research Gaps	5
1.5. Analysis from Literature Review.....	5
1.5.1 Comparative Evaluation.....	5
1.5.2 Analytical Discussion	6
1.6. Methodology and Software Lifecycle for This Project	6
1.6.1 Rationale behind Selected Methodology	7
CHAPTER 2: PROBLEM DEFINITION	8
2.1 Problem Statement.....	8
2.1.1 Context of the Problem.....	8
2.1.2 Identified Challenges.....	8
2.1.3 Core Problem Definition	9
2.1.4 Expected Outcome.....	9
2.2 Deliverables and Development Requirements.....	9
2.2.1 Project Deliverables	9
2.2.2 Development Requirements	10
2.3 Current System (if applicable to your project)	11
2.3.1 Overview of Existing Systems	11
2.3.2 Example of an Existing System.....	11
2.3.3 Comparative Analysis of Existing vs. Proposed System.....	12
2.3.4 Observations.....	13
2.3.5 Summary of Chapter.....	13
CHAPTER 3: REQUIREMENT ANALYSIS.....	14
3.1 Use Cases.....	14
3.1.1 Use Case Diagram	15
3.1.2 Use Case description.....	16
3.2 Non-Functional Requirements.....	32

CHAPTER 4: DESIGN AND ARCHITECTURE	34
4.1 UML Structural Diagrams	34
4.1.1 <i>Component Diagram</i>	34
4.1.2 <i>Package Diagram</i>	35
4.1.3 <i>Deployment Diagram</i>	36
4.1.4 <i>Class Diagram</i>	37
4.1.5 <i>Block Diagram</i>	38
4.2 UML Behavioral Diagrams	38
4.2.1 <i>Activity Diagram</i>	39
4.3 UML Interaction Diagram	42
4.3.1 <i>Sequence Diagram</i>	43
4.3.2 <i>Collaboration Diagram</i>	46
CHAPTER 5: IMPLEMENTATION	49
5.1 Communication Protocol Implementation	49
5.1.1 <i>Component Diagram</i>	49
5.2 Network and Protocol Choice	51
5.2.1 <i>Communication Protocols</i>	51
5.2.2 <i>Security Considerations</i>	52
5.2.3 <i>Performance Optimization</i>	52
5.3 Choice of Object Middleware	53
5.3.1 <i>User Management Module</i>	53
5.3.2 <i>Ticket Booking Module</i>	54
5.3.3 <i>Real Time Travel Insights Module</i>	54
5.4 User Interface	55
CHAPTER 6: TESTING AND EVALUATION	62
6.1 Verification	62
6.1.1 <i>User Module Verification</i>	62
6.1.2 <i>Ticket Booking & Route Module Verification</i>	62
6.1.3 <i>Provider & Notification Module Verification</i>	62
6.1.4 <i>AI Module Verification</i>	62
6.2 Validation	63
6.2.1 <i>User Validation</i>	63
6.2.2 <i>Functional Validation</i>	63
6.2.3 <i>Performance Validation</i>	63
6.3 Usability Testing	63
6.3.1 <i>User Interface Usability</i>	63
6.3.2 <i>Ticket Booking Usability</i>	63
6.3.3 <i>Route & Notification Usability</i>	64
6.3.4 <i>Provider Dashboard Usability</i>	64
6.4 Unit Testing	64
6.5 Integration Testing	72
6.5.1 <i>User Login & Ticket Booking Integration</i>	72
6.5.2 <i>Ticket Booking & Routes Integration</i>	72
6.5.3 <i>AI Fraud Detection & Booking Integration</i>	72
6.5.4 <i>Provider Management & Booking Integration</i>	73
6.5.5 <i>Notification & Booking Integration</i>	73
6.5.6 <i>Database & Backend Integration</i>	73
6.6 System Testing	73
6.6.1 <i>User Registration & Login Module</i>	73
6.6.2 <i>Ticket Booking Module</i>	73
6.6.3 <i>Route & Map Module</i>	73
6.6.4 <i>Provider Management Module</i>	74
6.6.5 <i>AI Module</i>	74

6.7	Acceptance Testing.....	74
6.7.1	<i>User Module Acceptance</i>	74
6.7.2	<i>Ticket Booking Acceptance</i>	74
6.7.3	<i>Route Map Acceptance</i>	74
6.7.4	<i>Provider Module Acceptance</i>	74
6.7.5	<i>AI Module Acceptance</i>	74
6.8	Stress Testing.....	75
6.8.1	<i>High User Load Testing</i>	75
6.8.2	<i>Multiple Booking Requests</i>	75
6.8.3	<i>Notification Load Testing</i>	75
6.8.4	<i>AI Processing Load</i>	75
6.9	Hardware Configuration for Testing.....	75
6.9.1	<i>Client-Side Hardware</i>	75
6.9.2	<i>Server-Side Hardware</i>	76
6.9.3	<i>Network Requirements</i>	76
6.10	Evaluation.....	76
6.10.1	<i>System Evaluation – Safarbot</i>	76
6.11	Deployment.....	77
6.12	Maintenance.....	77
CHAPTER 7: CONCLUSION AND FUTURE WORK.....		78
7.1	Conclusion.....	78
7.2	Future Work	78

List of Figures

Figure 2.1 : Sample Picture	12
Figure 3.1: Use Case Diagram	15
Figure 4.1: Component Diagram	34
Figure 4.2:Package Diagram	35
Figure 4.3:Deployment Diagram	36
Figure 4.4:Class Diagram	37
Figure 4.5:Block Diagram	38
Figure 4.6:Activity Diagram: User Registration	39
Figure 4.7:Activity Diagram: Admin Manage Provider.....	40
Figure 4.8:Activity Diagram: Booking Ticket.....	41
Figure 4.9:Activity Diagram: Real Time Travel Insights.....	42
Figure 4.10:Sequence Diagram: Admin Manage User.....	43
Figure 4.11:Sequence Diagram: User Registration/Login.....	44
Figure 4.12:Sequence Diagram: Booking Ticket	44
Figure 4.13:Sequence Diagram: Provider Manage Routes.....	45
Figure 4.14:Sequence Diagram: Real Time Travel Updates	45
Figure 4.15:Collaboration Diagram: User registration/login.....	46
Figure 4.16:Collaboration Diagram: Admin Manage User	46
Figure 4.17:Collaboration Diagram: Booking Ticket.....	47
Figure 4.18:Collaboration Diagram: Real Time Travel Updates	47
Figure 4.19:Collaboration Diagram: Provider Manage Routes	48
Figure 5.1:Component Diagram	50
Figure 5.2:Home Page	55
Figure 5.3:About Page	55
Figure 5.4:View Bookings.....	56
Figure 5.5:Contact Page.....	56
Figure 5.6:Provider Dashboard.....	57
Figure 5.7:Routes Manage.....	57
Figure 5.8:Fraud Alert	58
Figure 5.9:Admin Dashboard	58
Figure 5.10:Manage Provider	59
Figure 5.11:Manage User	59
Figure 5.12:View Reports.....	60
Figure 5.13:Register Page.....	60
Figure 5.14>Login Page	61

List of Table

Table 1.1: Course Integration	3
Table 1.2: Comparison with existing system.....	6
Table 2.1: Comparison between Existing and Proposed Systems.....	12
Table 3.1 : UC-1.0	16
Table 3.2: UC-2.0	17
Table 3.3: UC-3.0	18
Table 3.4: UC-4.0	19
Table 3.5: UC-5.0	20
Table 3.6: UC-6.0	21
Table 3.7: UC-7.0	22
Table 3.8: UC-8.0	23
Table 3.9: UC-9.0	24
Table 3.10: UC-10.0	25
Table 3.11: UC-11.0	26
Table 3.12: UC-12.0	27
Table 3.13: UC-13.0	28
Table 3.14: UC-14.0	29
Table 3.15: UC-15.0	30
Table 3.16: Functional Requirements Table.....	31
Table 3.17: Non Functional Requirements Table.....	33
Table 6.1: User Registration with Valid Details.....	64
Table 6.2: User Login with Valid Credentials.....	65
Table 6.3: Route Search with Valid Details	65
Table 6.4: Seat Availability Validation	66
Table 6.5: Ticket Booking with Valid Data.....	67
Table 6.6: Stripe Test Payment Success	67
Table 6.7: Voice Booking Input Processing.....	68
Table 6.8: AI Fraud Detection for Duplicate Booking.....	69
Table 6.9: Real-Time Travel Insight Retrieval.....	69
Table 6.10: Provider Route Creation with Valid Data	70
Table 6.11: Booking Confirmation Notification Generation.....	71
Table 6.12: Admin Fraud Alert View.....	71

CHAPTER 1: INTRODUCTION

This chapter gives a complete overview of the SafarBot project — a smart, AI-powered web application designed to make online ticket booking easier, especially for people who are uneducated or less familiar with digital systems. It explains the background, goals, methodology, and connection to the subjects studied during the degree.

1.1. Brief

This brief introduces the purpose, expected outcomes, technologies, and development approach of SafarBot. It gives a clear foundation for understanding how the system combines ticket booking, AI support, voice interaction, fraud detection, and real-time travel insights.

1.1.1. *Overview of the Project*

SafarBot is an AI-integrated web-based transport ticket booking system developed using the MERN stack. The system enables users to book intercity transport tickets through voice or text commands in both Urdu and English, making the platform accessible to a wider audience, including elderly and less-educated users.

The platform incorporates AI-powered fraud detection and real-time travel insights to assist users throughout the booking process. SafarBot aims to eliminate complexity, enhance security, and improve accessibility in online ticket booking systems.

1.1.2. *Project Outcomes*

- Voice-enabled ticket booking system (Urdu & English).
- AI-based fraud detection for suspicious bookings.
- Real-time travel insights (traffic & weather)
- Smart route suggestions for faster and safer travel.
- Secure and transparent digital payment system.
- Accessible platform for low-literacy users.
- Improved customer experience through faster and reliable ticket booking.

1.1.3. Tools and Technologies Used

The project uses modern software tools and technologies:

- ***Front-end:*** React.js, HTML5, CSS3, JavaScript
- ***Back-end:*** Node.js, Express.js
- ***Database:*** MongoDB (MongoDB Atlas)
- ***AI Components:*** Python, FastAPI, Hugging Face Models (Fraud Detection & Insights)
- ***Voice Processing:*** Speech-to-Text APIs
- ***Payment Gateway:*** Stripe Test Mode
- ***Deployment:*** Vercel, Render

1.1.4. Methodology Overview

The system is being developed using the Agile Software Development Methodology, which divides the project into short, iterative sprints. Each sprint focuses on developing a specific functional module and is followed by review and user feedback to ensure continuous improvement.

1.2. Relevance to Course Modules

This section connects SafarBot with the academic courses studied during the BS Software Engineering degree. It shows how concepts from artificial intelligence, web engineering, databases, project management, software architecture, quality assurance, and HCI are applied in a practical system.

1.2.1 Application of Academic Knowledge

This Final Year Project (FYP) is directly related to several core and elective courses studied during the BS Software Engineering degree. It demonstrates how theoretical knowledge gained throughout the program is applied to solve real-world problems.

1.2.2 Course Integration

The table below explains how each academic course contributes to the development of SafarBot. It links course concepts with practical implementation areas such as AI, backend development, database design, testing, and user-interface design.

Table 1.2: Course Integration

Course Name	Relevance to Project
Artificial Intelligence	Development of machine learning models for fraud detection and predictive insights
Web Engineering & Mobile App Management	Design and implementation of front-end and back-end web modules.
Database Systems	Structuring, storing, and managing booking and user data management.
Software Project Management	Sprint planning, Agile process management, and risk assessment.
Software Design & Architecture	Modular and scalable system architecture using object-oriented principles.
Software Quality Assurance	Testing of system functionality, performance, and reliability.
Human-Computer Interaction (HCI)	Designing an intuitive, user-friendly interface for users.

1.2.3 Professional Skill Development

The project enhances problem-solving, teamwork, version control, and real-world analytical thinking—skills vital for a software engineer. It bridges the gap between classroom learning and practical field implementation.

1.3. Project Background

The background explains why SafarBot was selected as a final year project and what real-world need it addresses. It focuses on the problems faced by users in existing booking systems and explains the motivation behind creating a smarter and more accessible solution.

1.3.1 Motivation

In Pakistan, online ticket booking platforms such as BookMe.pk and Bookaru are widely used but remain inaccessible to many individuals due to language barriers, complex interfaces, and limited automation. Elderly and low-literacy users often struggle with text-heavy systems and lack of real-time assistance.

This challenge motivated the development of SafarBot—an AI-driven, voice-enabled booking system designed to simplify ticket booking while ensuring security and inclusivity.

1.3.2 Problem Context

Existing booking systems focus mainly on manual searches and payment processing, offering limited real-time insights and weak fraud prevention. Additionally, the absence of voice-based interaction restricts usability for a large segment of the population. SafarBot addresses these issues by integrating AI automation, fraud detection, and real-time intelligence into a single platform.

1.3.3 Project Vision

The vision of SafarBot is to create an intelligent and inclusive transport booking ecosystem where users can interact naturally through voice, receive instant assistance, and complete secure bookings with confidence. The platform aims to bridge the gap between technology and accessibility across all social groups.

1.4. Literature Review

This section reviews existing online ticket booking systems and modern technology trends related to digital reservations. The review helps identify the gaps that SafarBot aims to fill through AI automation, bilingual voice interaction, and real-time travel support.

1.4.1 Existing Systems and Research

- ***BookMe.pk***: Provides online ticket booking but lacks AI automation and voice support
- ***Bookaru***: Offers basic transport booking with limited intelligence

1.4.2 Current Trends

Modern booking platforms emphasize:

- Conversational AI and chatbots.
- Real-time travel and traffic insights.
- AI-based fraud prevention.
- Cloud-hosted scalable architectures.
- Accessibility-focused UI/UX design.

1.4.3 Identified Research Gaps

Most existing platforms lack:

- Voice-based bilingual interaction
- AI-driven fraud detection
- Integrated real-time travel intelligence
- Inclusive design for low-literacy users

1.5. Analysis from Literature Review

This analysis summarizes the main findings from the reviewed systems and technologies. It compares existing applications with SafarBot to show how the proposed system improves accessibility, automation, security, and user assistance.

1.5.1 Comparative Evaluation

The following comparison highlights the key differences between existing ticket booking applications and the proposed SafarBot system. It shows how SafarBot extends basic booking functionality by adding voice support, AI assistance, fraud detection, and travel insights. Additionally, this evaluation demonstrates how the proposed system improves accessibility, security, and overall user experience compared to traditional platforms.

Table 1.5: Comparison with existing system

<i>Feature</i>	<i>Existing Apps</i>	<i>SafarBot (Proposed)</i>
Online Booking	Yes	Yes
Voice Interaction	No	Yes
AI Chatbot	Limited	Yes
Fraud Detection	No	Yes
Real-Time Insights	Limited	Yes

1.5.2 Analytical Discussion

The analysis indicates that current systems prioritize booking functionality while neglecting automation, security, and inclusivity. SafarBot introduces AI-powered voice interaction, fraud detection, and real-time intelligence, creating a smarter and more user-centric booking solution aligned with modern technological trends.

1.6. Methodology and Software Lifecycle for This Project

The Agile Software Development Life Cycle (SDLC) has been adopted for this project due to its iterative and adaptive nature. It divides development into multiple sprints—each delivering incremental progress while maintaining flexibility to incorporate user feedback.

Implementation Strategy

Each sprint will focus on a core functionality:

1. ***Sprint 1:*** Requirement gathering and UI/UX prototyping
2. ***Sprint 2:*** User authentication and booking modules
3. ***Sprint 3:*** Voice interaction integration
4. ***Sprint 4:*** AI fraud detection and real-time insights
5. ***Sprint 5:*** Testing, optimization, and deployment

Benefits of Agile for This Project

- Continuous testing and improvement
- Rapid identification of issues
- Flexibility for design and AI model enhancements
- Regular client/supervisor feedback integration

1.6.1 Rationale behind Selected Methodology

The Agile model was selected because:

- The system involves evolving requirements (especially for AI and GIS integration).
- It allows continuous feedback from supervisors and potential end-users.
- Each feature can be tested independently, ensuring stability.
- The iterative process supports parallel development of AI, web, and database modules

1.6.1.1 Rationale behind System Architecture Design

The system architecture follows Object-Oriented and modular principles:

- Reusable components such as User, Booking, Payment, and FraudModel
- Improved scalability for future enhancements
- Easier maintenance and feature expansion

Combining Agile methodology with modular system design provides both development flexibility and architectural robustness, making it suitable for an AI-driven, real-time booking platform like SafarBot.

CHAPTER 2: PROBLEM DEFINITION

This chapter defines the core problem addressed by SafarBot. It explains the nature of issues present in existing online ticket booking systems in Pakistan, the objectives of solving these issues, and the project deliverables. It also discusses current systems, highlighting their limitations and how the proposed SafarBot system overcomes them.

2.1 Problem Statement

This section defines the main issue addressed by SafarBot. It explains why many users face difficulty with existing online booking platforms and how these limitations affect accessibility, trust, and operational efficiency.

2.1.1 *Context of the Problem*

In Pakistan, online ticket booking platforms such as BookMe.pk and Bookaru are widely used for intercity transport reservations. However, these systems are primarily designed for users who are literate, technologically skilled, and comfortable with English-based, text-heavy interfaces.

A large segment of the population—including elderly individuals, uneducated users, and people with language barriers—faces difficulties in searching routes, entering information, and completing bookings. Additionally, existing platforms lack intelligent automation, real-time travel updates, and effective fraud prevention mechanisms, which reduces user trust and system reliability.

2.1.2 *Identified Challenges*

- Text-based interfaces that are difficult for uneducated and elderly users
- No support for voice-based booking or Urdu language interaction
- Absence of AI-based fraud detection for fake or duplicate bookings
- Lack of real-time updates related to traffic, weather, or travel delays
- Manual and slow customer support services
- No safe Stripe test mode environment for testing payment workflows

2.1.3 Core Problem Definition

The current online ticket booking systems in Pakistan are inaccessible, insecure, and poorly automated. These limitations result in usability issues, customer dissatisfaction, and operational inefficiencies for transport providers.

SafarBot aims to address these problems by developing an AI-powered, voice-enabled booking platform capable of:

- Voice and text-based ticket booking in Urdu and English
- AI-based fraud detection to identify suspicious bookings
- Real-time travel insights including weather and traffic conditions
- Automated customer assistance through intelligent AI support
- A secure and accessible booking environment for all users

2.1.4 Expected Outcome

The expected outcome of the SafarBot project is a smart, secure, and inclusive ticket booking system that improves accessibility, trust, and efficiency. The system will reduce booking errors, prevent fraud, enhance user experience, and provide transport providers with better operational control.

2.2 Deliverables and Development Requirements

This section outlines the main outputs of the SafarBot project and the resources required to build them. It includes the working software modules, supporting documentation, hardware, software, development tools, and team requirements.

2.2.1 Project Deliverables

The deliverables of the SafarBot project include both functional software components and supporting documentation.

Main Deliverables:

1. ***Web Application Platform*** – MERN stack-based system for transport ticket booking
2. ***AI Voice Agent*** – Voice-based booking using Urdu and English commands
3. ***AI Fraud Detection Module*** – Identification of fake, duplicate, or suspicious bookings.

4. ***AI Real-Time Insights System*** – Live updates on traffic, weather, and delays.
5. ***Payment Sandbox Integration – Stripe test mode environment for secure payment demonstrations.***
6. ***Admin & Provider Panels*** – Route management, booking monitoring, and fraud alerts
7. ***Documentation*** –
 - Software Requirement Specification (SRS)
 - System Design Diagrams (UML, ERD, Use Case)
 - Testing and Evaluation Report
 - Final Project Report and Presentation

2.2.2 Development Requirements

Hardware Requirements:

- Laptop or PC with minimum 8 GB RAM
- 250 GB storage or higher
- Stable internet connection
- Microphone for voice input testing

Software Requirements:

- Frontend: React.js, Tailwind CSS
- Backend: Node.js, Express.js
- Database: MongoDB Atlas
- AI Libraries: TensorFlow, Scikit-learn, Hugging Face, Transformers,
- Payment Sandbox: Stripe Test Mode
- Deployment: Vercel (Frontend), Render/Heroku (Backend), Hugging Face (AI Models)

Development Tools:

- Visual Studio Code
- Postman (API Testing)
- Figma (UI/UX Design)

- Google Colab
- GitHub (Version Control)

Team Requirements:

- 3 Developers (team members)
- 1 Supervisor for technical guidance

2.3 Current System (if applicable to your project)

This section explains how existing ticket booking systems currently operate and where they fall short. It provides a comparison point for understanding the improvements introduced by SafarBot.

2.3.1 Overview of Existing Systems

Current online ticket booking systems in Pakistan, such as BookMe.pk and Bookaru, allow users to reserve seats for buses, trains, and events. These platforms focus mainly on booking and payment functionality but lack intelligent automation and accessibility features.

Limitations of Existing Systems:

- Entirely text-based user interaction
- Manual customer support
- No voice or Urdu language support
- No AI-based fraud prevention
- No real-time travel alerts

2.3.2 Example of an Existing System

Figure 2.1 illustrates a sample interface of a current online ticket booking system, specifically BookMe.pk, highlighting its text-heavy layout and limited accessibility. The interface relies primarily on English text and manual input, which can be challenging for users with low literacy or limited technical skills. It also lacks voice interaction and intelligent assistance features, making the overall user experience less inclusive and harder to navigate for a broader audience.

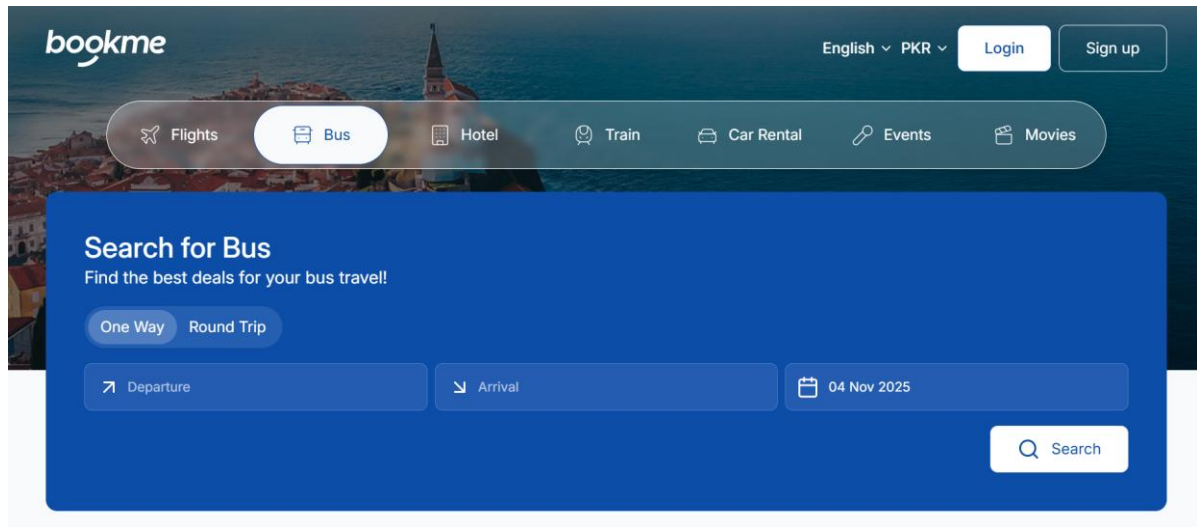


Figure 2.3 : Sample Picture

2.3.3 Comparative Analysis of Existing vs. Proposed System

The following table compares existing booking platforms with the proposed SafarBot system, highlighting improvements in user interaction, security, and accessibility.

Table 2.3.3: Comparison between Existing and Proposed Systems

Feature	BookMe / Bookaru	SafarBot
User Interaction	Text only	Voice + Text (Urdu/English)
Fraud Detection	Not available	AI-based fraud detection
Real-Time Insights	Not available	Weather & traffic-based insights
Customer Support	Manual chat	AI chatbot with auto-escalation
Payment Integration	Live Payments	Stripe Test Mode (safe demo)
Accessibility	For educated users only	For all, including uneducated users

2.3.4 Observations

The comparison clearly shows that existing systems are functional but lack intelligence, accessibility, and security. SafarBot enhances the booking experience by integrating AI-driven automation, voice interaction, fraud detection, and real-time insights into a single unified platform. This transformation shifts ticket booking from a manual and restrictive process to a smart and user-centric system.

2.3.5 Summary of Chapter

This chapter identified the main problems in existing online ticket booking systems in Pakistan, including accessibility issues, lack of automation, and security concerns. It outlined the proposed solution—SafarBot—along with its deliverables, development requirements, and comparison with current systems. The next chapter will present a detailed Literature Review, analyzing existing research and technologies related to AI-based booking and voice-enabled systems.

CHAPTER 3: REQUIREMENT ANALYSIS

This chapter outlines the *Software Requirements Specification (SRS) for SafarBot*. It identifies the system's use cases, actors, and their interactions with the platform. The SRS serves as a foundation for system design, development, implementation, and testing throughout the software development lifecycle.

SafarBot is an AI-powered, voice-enabled transport ticket booking system designed to improve accessibility, security, and efficiency in online ticket booking, especially for users in Pakistan.

3.1 Use Cases

Use cases define the functional interactions between the system and its users (actors), illustrating how specific goals are achieved through a sequence of actions. In the SafarBot platform, multiple actors are involved, each with distinct roles and responsibilities, including Passenger (Customer), Service Provider, and Administrator. These actors interact with the system to perform operations that collectively support the core functionality of the platform.

Passengers primarily use the system to register and authenticate, search available routes and schedules, book tickets, make payments, manage bookings, and receive real-time notifications. Additionally, passengers may interact with AI-driven features such as chatbot or voice-based assistance to obtain travel information and support.

Service Providers interact with SafarBot to create and manage routes, define schedules, set pricing, monitor bookings, and analyze operational insights through reports and dashboards. These actions enable providers to efficiently manage transportation services and optimize resource utilization.

Administrators oversee the overall system operations by managing users and providers, monitoring bookings and transactions, handling fraud alerts, and enforcing system policies and permissions. The admin role ensures system security, data integrity, and smooth platform operation.

Together, these use cases provide a comprehensive view of system functionality from the perspective of each actor, serving as a foundation for requirement analysis, system design,

and validation. By clearly defining actor interactions, the use-case model helps ensure that SafarBot meets functional requirements while supporting scalability, security, and intelligent, real-time booking operations.

The following use cases capture the major functionalities and interactions that define the overall behavior of the system.

3.1.1 Use Case Diagram

The use case diagram provides a visual overview of the main actors and their interactions with the system. It helps readers understand how passengers, service providers, and administrators use SafarBot to complete their tasks.

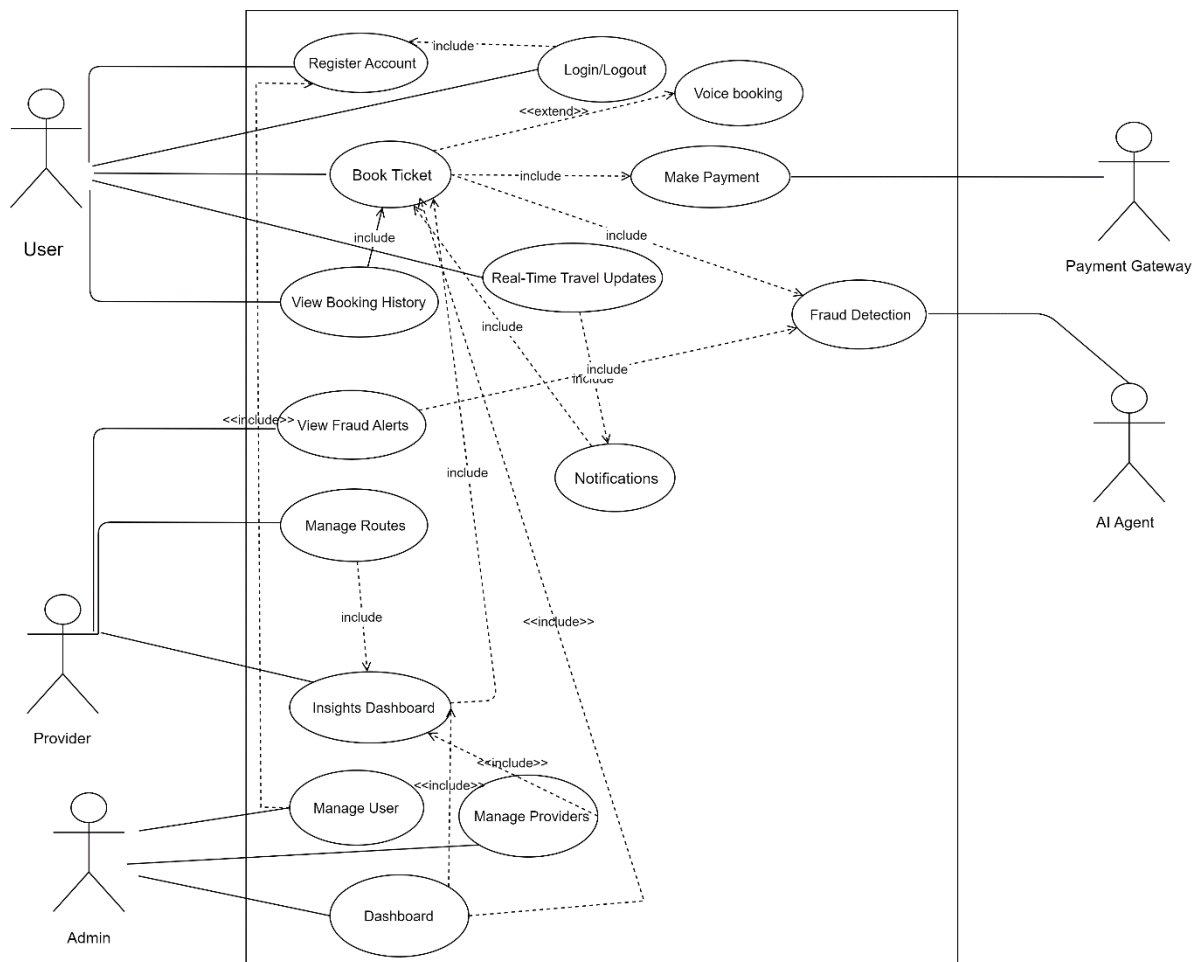


Figure 3.1: Use Case Diagram

3.1.2 Use Case description

The following use case descriptions explain each major system function in a structured format. They define the actors, triggers, normal flows, exceptions, assumptions, and expected outcomes for the main features of SafarBot.

Table 3.1.2 : UC-1.0

Use Case ID:	UC-1.0
Use Case Name:	Register Account
Actors:	User, System Database
Description:	Enables a new user to create an account in the SafarBot system by providing basic personal information such as name, email, and password. The system validates the input and stores it securely in the database.
Trigger:	User selects the “Register” option.
Preconditions:	User is not already registered and has an active internet connection.
Postconditions:	A new user account is created and a confirmation message is displayed.
Normal Flow:	User opens the registration page. User enters required personal details. System validates the entered data. System stores the information in the database. System confirms successful registration.
Alternative Flows:	If the email already exists, the system prompts the user to log in.

Exceptions:	Network or database failure during registration.
Includes:	None.
Special Requirements:	Secure password storage.
Assumptions:	User has a valid email address.
Notes and Issues:	None.

Table 3.1.2: UC-2.0

Use Case ID:	UC-2.0
Use Case Name:	Login/Logout
Actors:	User, Provider, Admin, Authentication System
Description:	Authenticates registered users and allows them to access or exit the system securely.
Trigger:	User enters login credentials or selects logout.
Preconditions:	User account exists in the system.
Postconditions:	User session is started or terminated.
Normal Flow:	User enters email and password. System verifies credentials. System grants access and redirects to dashboard.
Alternative Flows:	Invalid credentials result in an error message.

Exceptions:	Authentication service unavailable.
Includes:	UC-1.0 (Register Account)
Special Requirements:	Secure session management.
Assumptions:	Credentials are valid.
Notes and Issues:	Multi-factor authentication may be added later.

Table 3.3: UC-3.0

Use Case ID:	UC-3.0
Use Case Name:	Book Ticket
Actors:	User, Provider system
Description:	Allows users to search available routes, select seats, and book tickets using the SafarBot system.
Trigger:	User initiates ticket booking.
Preconditions:	User must be logged in.
Postconditions:	Ticket booking is confirmed and stored.
Normal Flow:	User searches for routes. User selects route and seat. System confirms booking details. User proceeds to payment.

Alternative Flows:	No seats available; system suggests alternatives.
Exceptions:	Route data unavailable.
Includes:	UC-2.0 (User Login)
Special Requirements:	Seat availability synchronization.
Assumptions:	Route data is up to date.
Notes and Issues:	High concurrency must be handled.

Table 3.4: UC-4.0

Use Case ID:	UC-4.0
Use Case Name:	Voice Booking
Actors:	User, AI Voice Agent
Description:	Enables users to book tickets using voice commands with AI assistance.
Trigger:	User initiates voice interaction.
Preconditions:	User is logged in and microphone access is available.
Postconditions:	Ticket is booked successfully.
Normal Flow:	User provides booking details via voice. AI interprets commands. System completes booking process.

Alternative Flows:	AI asks user to repeat unclear input.
Exceptions:	Voice recognition failure.
Extends:	UC-3.0 (Book Ticket)
Special Requirements:	Speech-to-text support.
Assumptions:	Clear voice input.
Notes and Issues:	Urdu accent accuracy needs testing.

Table 3.5: UC-5.0

Use Case ID:	UC-5.0
Use Case Name:	Make Payment
Actors:	User, Payment Gateway
Description:	Processes ticket payment through an external payment gateway.
Trigger:	User confirms booking.
Preconditions:	Valid booking exists.
Postconditions:	Payment is completed or declined.
Normal Flow:	User selects payment method. System redirects to payment gateway. Transaction is processed. System confirms payment status.

Alternative Flows:	User cancels payment.
Exceptions:	Gateway timeout or failure.
Includes:	UC-3.0 (Book Ticket)
Special Requirements:	Secure transaction handling.
Assumptions:	Payment gateway is operational.
Notes and Issues:	Sandbox used during development.

Table 3.6: UC-6.0

Use Case ID:	UC-6.0
Use Case Name:	View Booking History
Actors:	User
Description:	Allows users to view previous and upcoming bookings.
Trigger:	User selects “Booking History.”
Preconditions:	User is logged in.
Postconditions:	Booking history is displayed.
Normal Flow:	User requests booking history. System retrieves records. System displays booking details.

Alternative Flows:	None.
Exceptions:	Database access error.
Includes:	None.
Special Requirements:	Pagination for large datasets.
Assumptions:	Booking records exist.
Notes and Issues:	None.

Table 3.7: UC-7.0

Use Case ID:	UC-7.0
Use Case Name:	Real-Time Travel Updates
Actors:	User, System
Description:	Provides live updates related to routes, delays, or schedule changes.
Trigger:	System detects change in travel status.
Preconditions:	Active booking exists.
Postconditions:	User receives updates.
Normal Flow:	System monitors route status. Update is detected. User is notified.

Alternative Flows:	None.
Exceptions:	None.
Includes:	UC-8.0 (Notifications)
Special Requirements:	Real-time data integration.
Assumptions:	External data sources are reliable.
Notes and Issues:	Duplicate alerts should be avoided.

Table 3.8: UC-8.0

Use Case ID:	UC-8.0
Use Case Name:	Send Notifications
Actors:	System
Description:	Sends automated alerts related to bookings, payments, and travel updates
Trigger:	System event occurs.
Preconditions:	Notification trigger exists.
Postconditions:	Notification is delivered.
Normal Flow:	Event is triggered. Notification is generated. User receives alert.

Alternative Flows:	None.
Exceptions:	Notification service unavailable.
Includes:	None.
Special Requirements:	Multi-channel support.
Assumptions:	User notifications are enabled.
Notes and Issues:	None.

Table 3.9: UC-9.0

Use Case ID:	UC-9.0
Use Case Name:	Fraud Detection
Actors:	AI Agent
Description:	Detects suspicious booking or payment activities using AI analysis.
Trigger:	Booking is initiated.
Preconditions:	Access to booking data.
Postconditions:	Fraud alert generated if needed.
Normal Flow:	AI analyzes booking. Suspicious patterns identified. Alert is generated.

Alternative Flows:	None.
Exceptions:	AI service unavailable.
Includes:	None.
Special Requirements:	ML model monitoring.
Assumptions:	Historical data exists.
Notes and Issues:	False positives possible.

Table 3.10: UC-10.0

Use Case ID:	UC-10.0
Use Case Name:	View Fraud Alerts
Actors:	Provider
Description:	Allows provider to view fraud alerts generated by the system.
Trigger:	Provider opens fraud alerts section.
Preconditions:	Provider is logged in.
Postconditions:	Fraud alerts displayed.
Normal Flow:	Provider requests fraud alerts. System retrieves alerts. Alerts are shown.

Alternative Flows:	None.
Exceptions:	None.
Includes:	UC-9.0 (Fraud Detection)
Special Requirements:	None.
Assumptions:	None.
Notes and Issues:	Alert prioritization may be added later.

Table 3.11: UC-11.0

Use Case ID:	UC-11.0
Use Case Name:	Manage Routes
Actors:	Provider
Description:	Enables providers to add, update, or remove bus routes.
Trigger:	Provider accesses route management.
Preconditions:	Provider is authenticated.
Postconditions:	Route data updated.
Normal Flow:	Provider adds or edits route. System validates data. Changes are saved.

Alternative Flows:	None.
Exceptions:	Invalid route information.
Includes:	None.
Special Requirements:	None.
Assumptions:	None.
Notes and Issues:	Admin approval may be required later.

Table 3.12: UC-12.0

Use Case ID:	UC-12.0
Use Case Name:	Insights Dashboard
Actors:	Provider
Description:	Displays analytics related to bookings, revenue, and system usage.
Trigger:	Provider opens dashboard.
Preconditions:	Data exists.
Postconditions:	Insights displayed.
Normal Flow:	Dashboard is accessed. Data is fetched. Charts and metrics are displayed.

Alternative Flows:	None.
Exceptions:	None.
Includes:	None.
Special Requirements:	None.
Assumptions:	None.
Notes and Issues:	AI predictions can be added later.

Table 3.13: UC-13.0

Use Case ID:	UC-13.0
Use Case Name:	Manage Users
Actors:	Admin
Description:	Allows admin to manage user accounts.
Trigger:	Admin selects user management.
Preconditions:	Admin is logged in.
Postconditions:	User records updated.
Normal Flow:	Admin views users. Admin edits or removes user. System confirms update.

Alternative Flows:	None.
Exceptions:	None.
Includes:	None.
Special Requirements:	None.
Assumptions:	None.
Notes and Issues:	Privacy compliance required.

Table 3.14: UC-14.0

Use Case ID:	UC-14.0
Use Case Name:	Manage Providers
Actors:	Admin
Description:	Allows admin to manage transport service providers.
Trigger:	Admin selects provider management.
Preconditions:	Admin authentication required.
Postconditions:	Provider data updated.
Normal Flow:	Admin views provider list. Admin updates provider details. System saves changes.

Alternative Flows:	None.
Exceptions:	None.
Includes:	None.
Special Requirements:	None.
Assumptions:	None.
Notes and Issues:	Provider verification needed.

Table 3.15: UC-15.0

Use Case ID:	UC-15.0
Use Case Name:	Admin Dashboard
Actors:	Admin
Description:	Provides a centralized interface for monitoring and controlling the SafarBot system.
Trigger:	Admin logs into dashboard.
Preconditions:	Admin is authenticated.
Postconditions:	System overview displayed.
Normal Flow:	Admin accesses dashboard. System loads statistics and controls. Admin monitors system status.

Alternative Flows:	None.
Exceptions:	None.
Includes:	None.
Special Requirements:	None.
Assumptions:	None.
Notes and Issues:	Role-based widgets may be added.

3.2 Functional Requirements

The **functional requirements** define the system behavior in response to user actions.

Table 3.16: Functional Requirements Table

ID	Requirement Description
FR-1	The system shall allow users to register and log in using valid credentials.
FR-2	The system shall allow users to book transport tickets using text-based input.
FR-3	The system shall allow users to book transport tickets using voice commands in Urdu and English.
FR-4	The system shall display available routes, schedules, and seat availability in real time.
FR-5	The system shall process ticket payments using a Stripe test mode payment gateway (Stripe Test Mode).

FR-6	The system shall provide booking confirmation and maintain booking history for users.
FR-7	The system shall send automated notifications for booking confirmation, cancellation, and travel updates.
FR-8	The system shall provide real-time travel insights including traffic, weather, and delay information.
FR-9	The system shall detect suspicious or fraudulent booking activities using AI-based models.
FR-10	The system shall display fraud alerts on the provider dashboard for review.
FR-11	The system shall allow transport providers to add, update, or remove routes and schedules.
FR-12	The system shall provide analytical dashboards for providers and administrators.
FR-13	The system shall allow administrators to manage users, providers, and system configurations.

3.2 Non-Functional Requirements

Non-functional requirements (NFRs) define how well a system performs rather than what it does. They focus on system quality attributes such as performance, reliability, scalability, security, and usability. NFRs also specify constraints like response time, system availability, data protection standards, and platform compatibility. These requirements ensure that the system operates efficiently under different conditions, provides a smooth user experience, and meets industry standards and user expectations.

Table 3.17: Non Functional Requirements Table

<i>Category</i>	<i>Requirement Description</i>
<i>Performance</i>	The system shall respond to user requests within 3 seconds under normal operating conditions.
<i>Scalability</i>	The system shall support at least 5,000 concurrent users during peak usage periods.
<i>Security</i>	All sensitive user data shall be encrypted, and authentication shall use JWT-based security mechanisms.
<i>Usability</i>	The user interface shall be intuitive and accessible for uneducated and elderly users.
<i>Reliability</i>	The system shall maintain a minimum uptime of 99% during operation.
<i>Maintainability</i>	The system shall use a modular architecture to allow easy maintenance and upgrades.
<i>Compatibility</i>	The system shall be compatible with modern web browsers and mobile devices.
<i>Availability</i>	The platform shall be available 24/7 using cloud-based deployment infrastructure.
<i>Accuracy</i>	The AI fraud detection module shall maintain an accuracy of at least 80% on test datasets.
<i>Localization</i>	The system shall support both Urdu and English for text and voice-based interaction.

CHAPTER 4: DESIGN AND ARCHITECTURE

This chapter explains the overall architecture and internal design of the *SafarBot*, including system components, interactions, and the design patterns used.

4.1 UML Structural Diagrams

UML Structural Diagrams describe what the system is made of, not how it behaves. UML Structural Diagrams represent the static layout and structure of a system and the relationships between its components.

4.1.1 Component Diagram

A UML diagram that shows the high-level system components and their dependencies, illustrating how different modules interact to form the complete system.

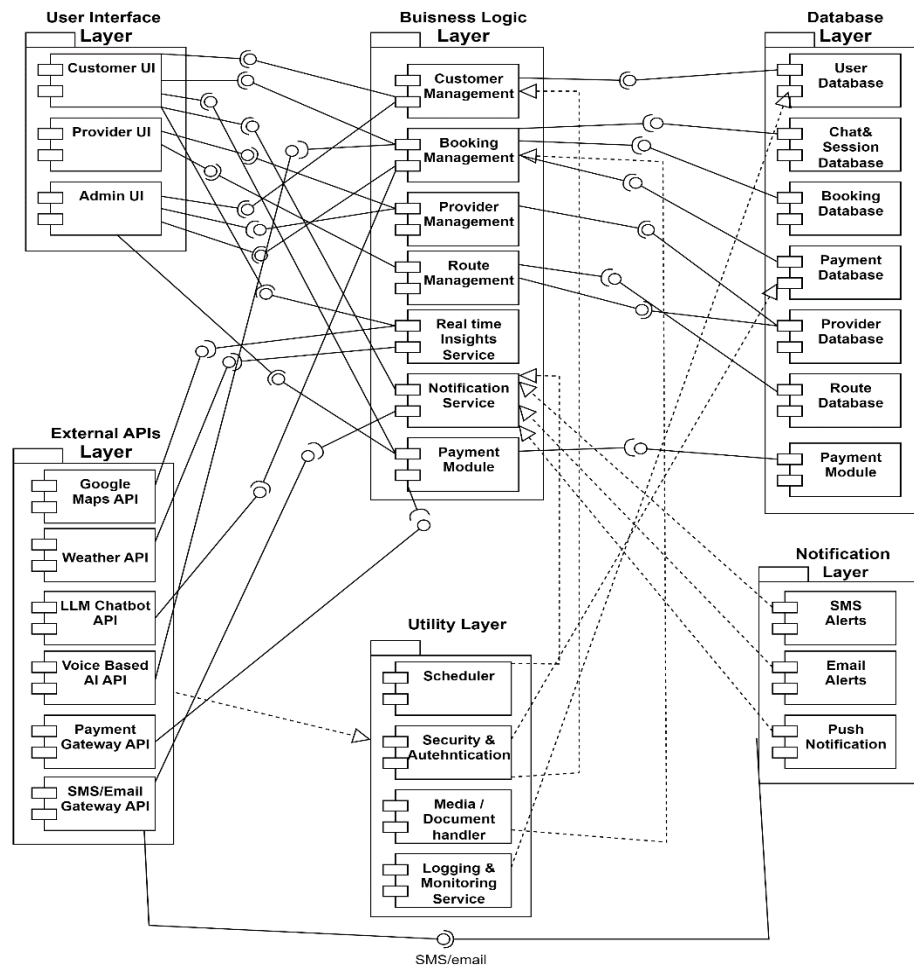


Figure 4.1: Component Diagram

4.1.2 Package Diagram

A UML diagram that organizes related classes and components into packages, showing the structural grouping and dependency relationships within the system.

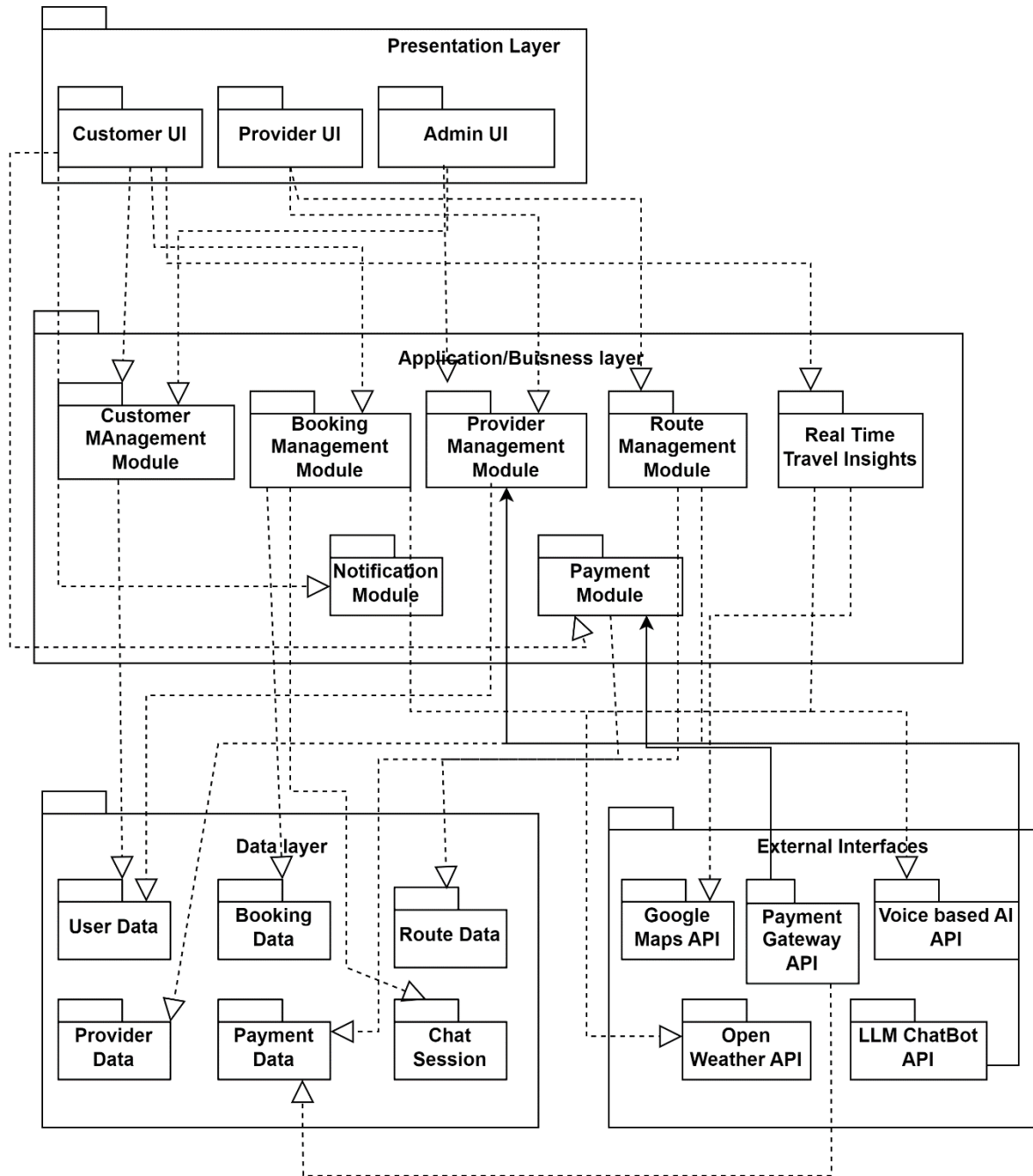


Figure 4.1:Package Diagram

4.1.3 Deployment Diagram

A Deployment Diagram is a UML diagram that shows the physical arrangement of hardware and software in a system, illustrating where system components are deployed and how they communicate.

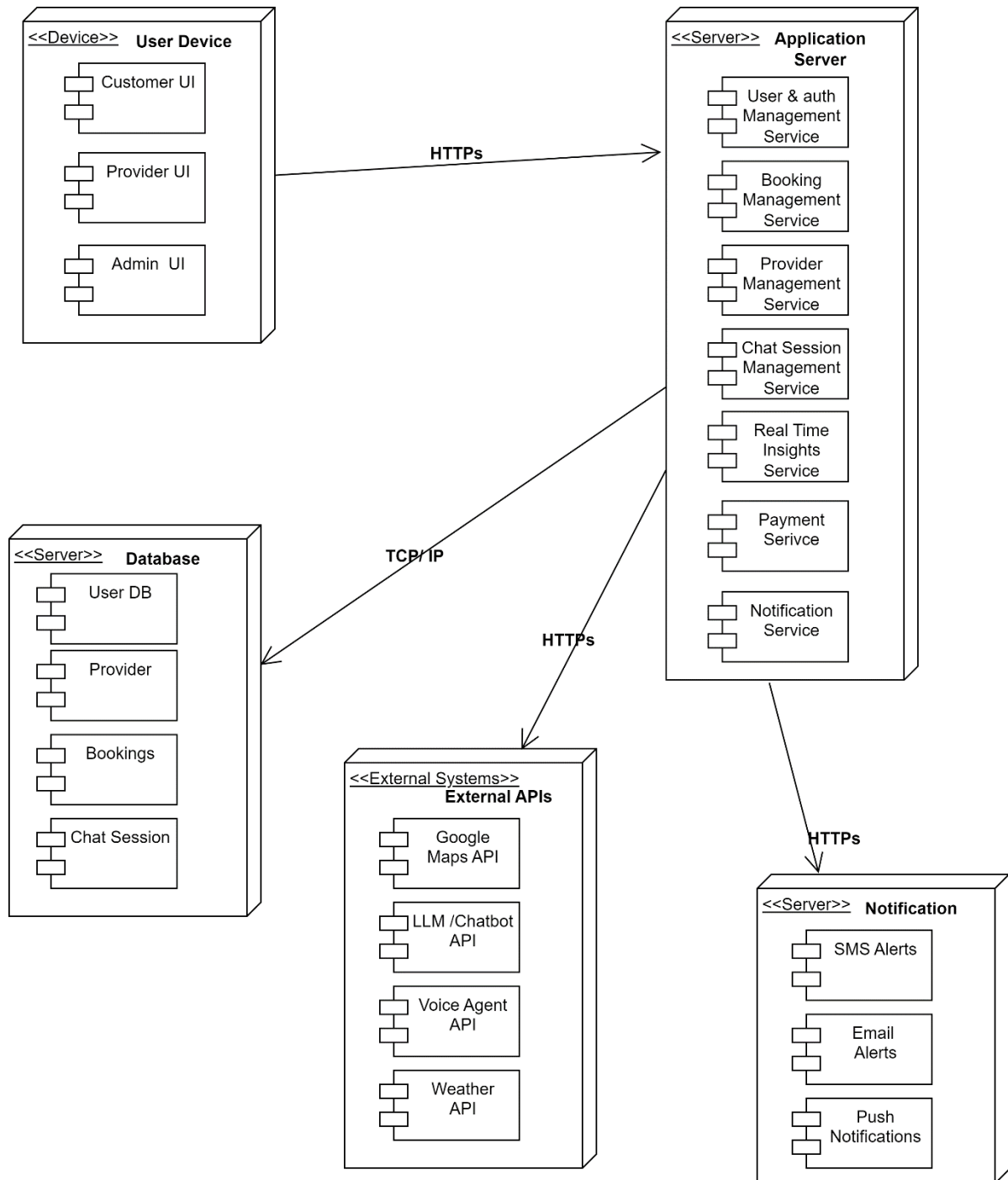


Figure 4.1:Deployment Diagram

4.1.4 Class Diagram

A Class Diagram is a UML diagram that shows the classes of a system along with their attributes, functions, and the relationships between them, helping to model the system's structure.

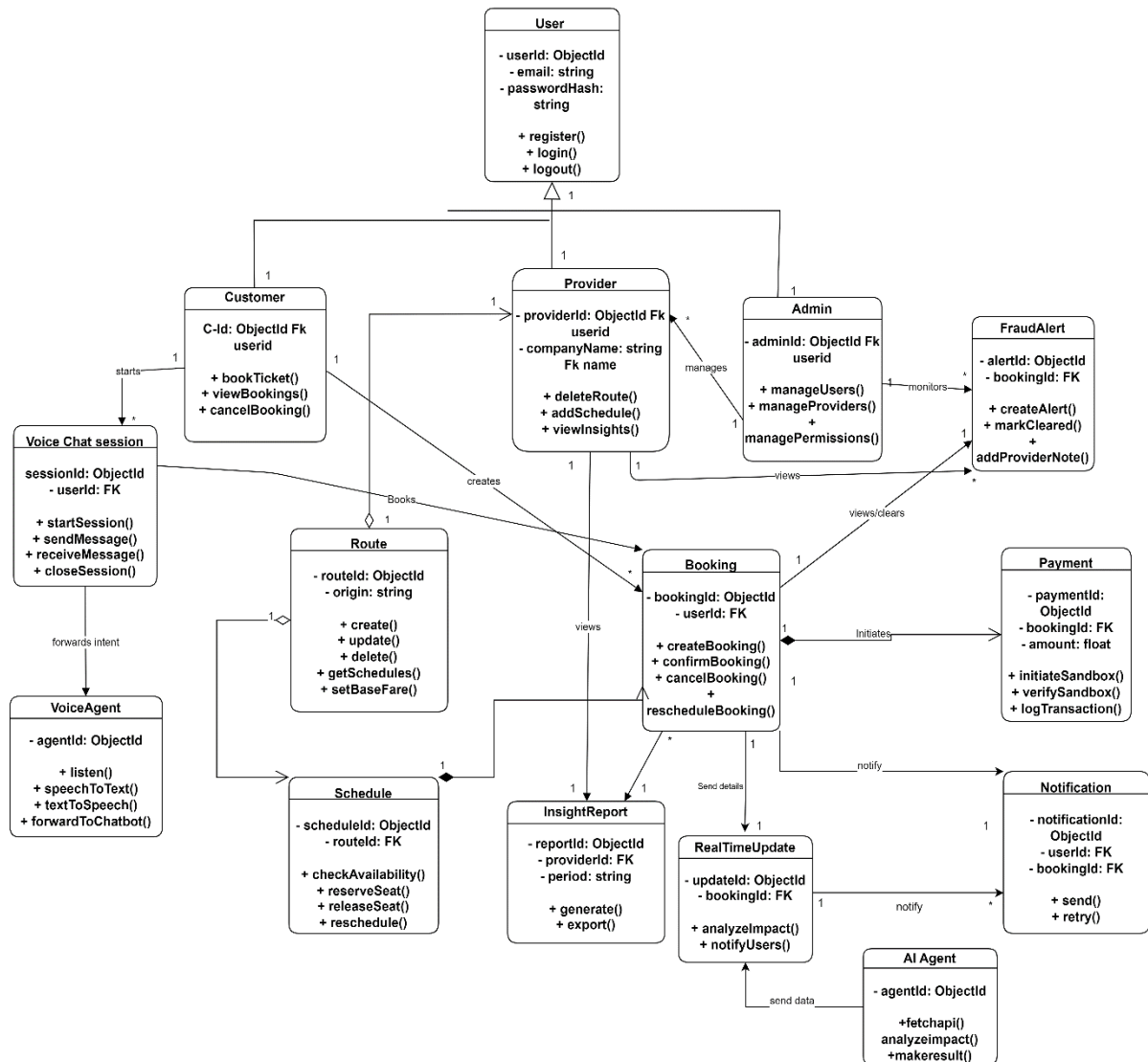


Figure 4.1:Class Diagram

4.1.5 Block Diagram

A block diagram is a simple and high-level visual representation of a system that illustrates its main components (blocks) and the relationships or connections between them. Each block represents a major function or subsystem rather than detailed internal workings. The connections between blocks show the flow of information, signals, data, or control from one component to another. Block diagrams are commonly used in engineering, computer science, and system design to provide a clear overview of how a system operates. They help designers, developers, and stakeholders quickly understand the system's structure without getting into technical details. Because of their simplicity, block diagrams are especially useful during the initial planning and analysis stages of a project.

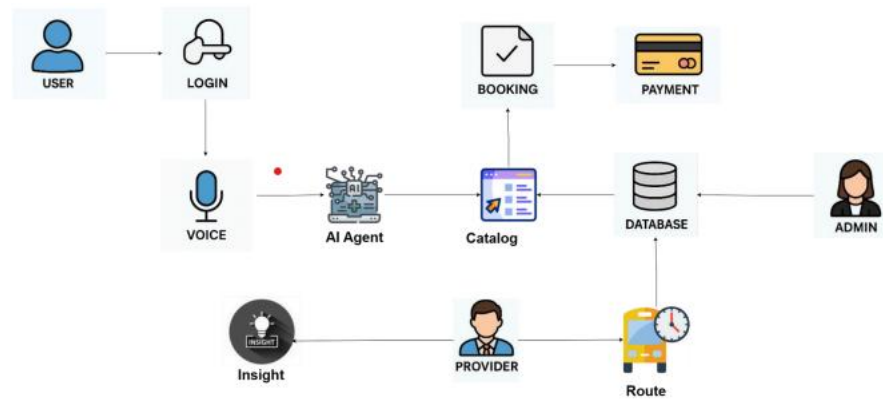


Figure 4.1:Block Diagram

4.2 UML Behavioral Diagrams

A UML Behavioral Diagram is used to represent the dynamic behavior of a system, showing how the system and its components interact, respond, and change over time. Unlike structural diagrams, which focus on what the system is made of, behavioral diagrams focus on how the system behaves when it is running. These diagrams illustrate user interactions, workflows, message exchanges, state changes, and control flow within the system.

4.2.1 Activity Diagram

An Activity Diagram is a UML behavioral diagram that visually represents the step-by-step flow of activities in a system, showing how processes begin, branch, merge, and end through actions, decisions, and parallel flows. It helps in understanding the dynamic behavior of the system by clearly illustrating workflow logic, control flow, and the sequence of operations. Additionally, activity diagrams are useful for analyzing business processes and use-case scenarios, making system behavior easier to communicate to both technical and non-technical stakeholders.

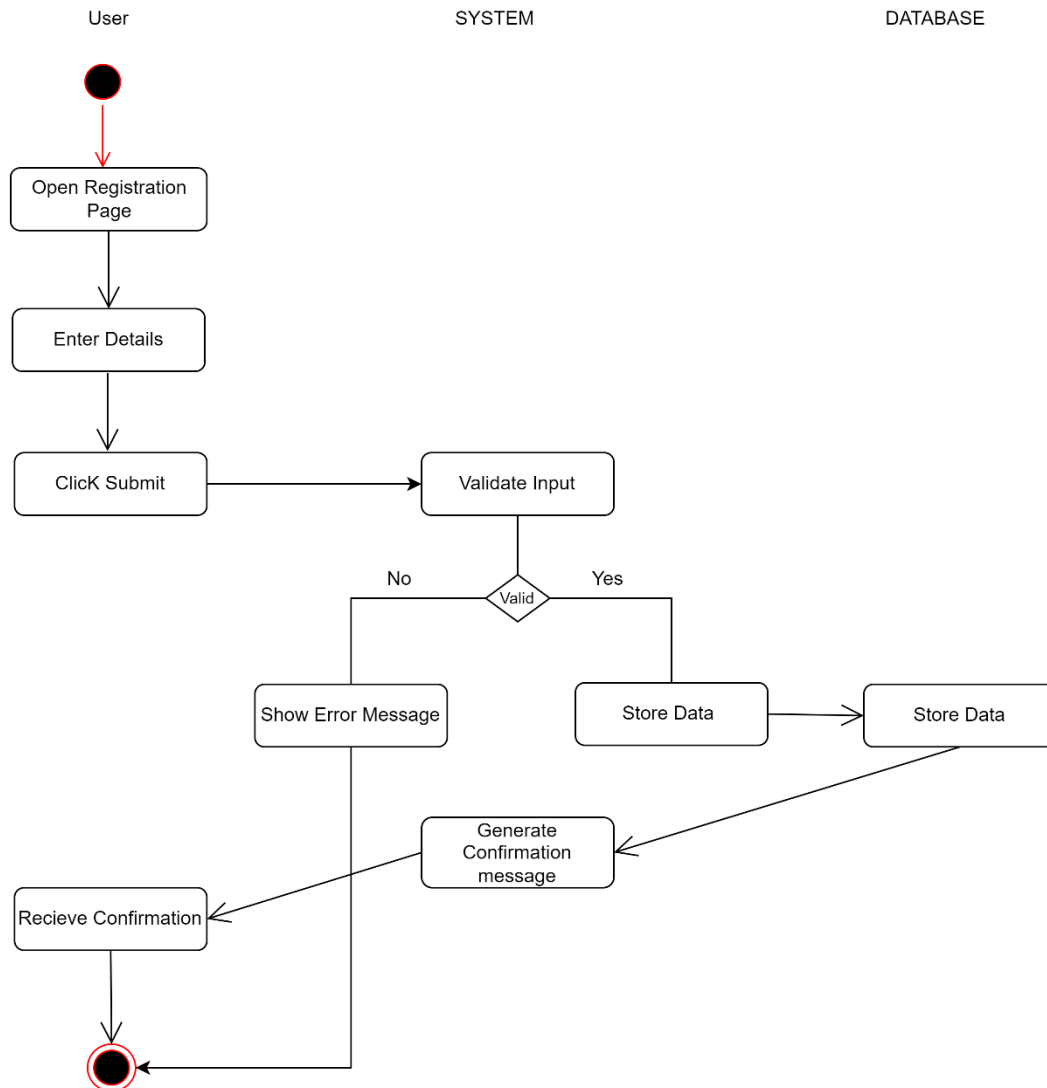


Figure 4.2:Activity Diagram: User Registration

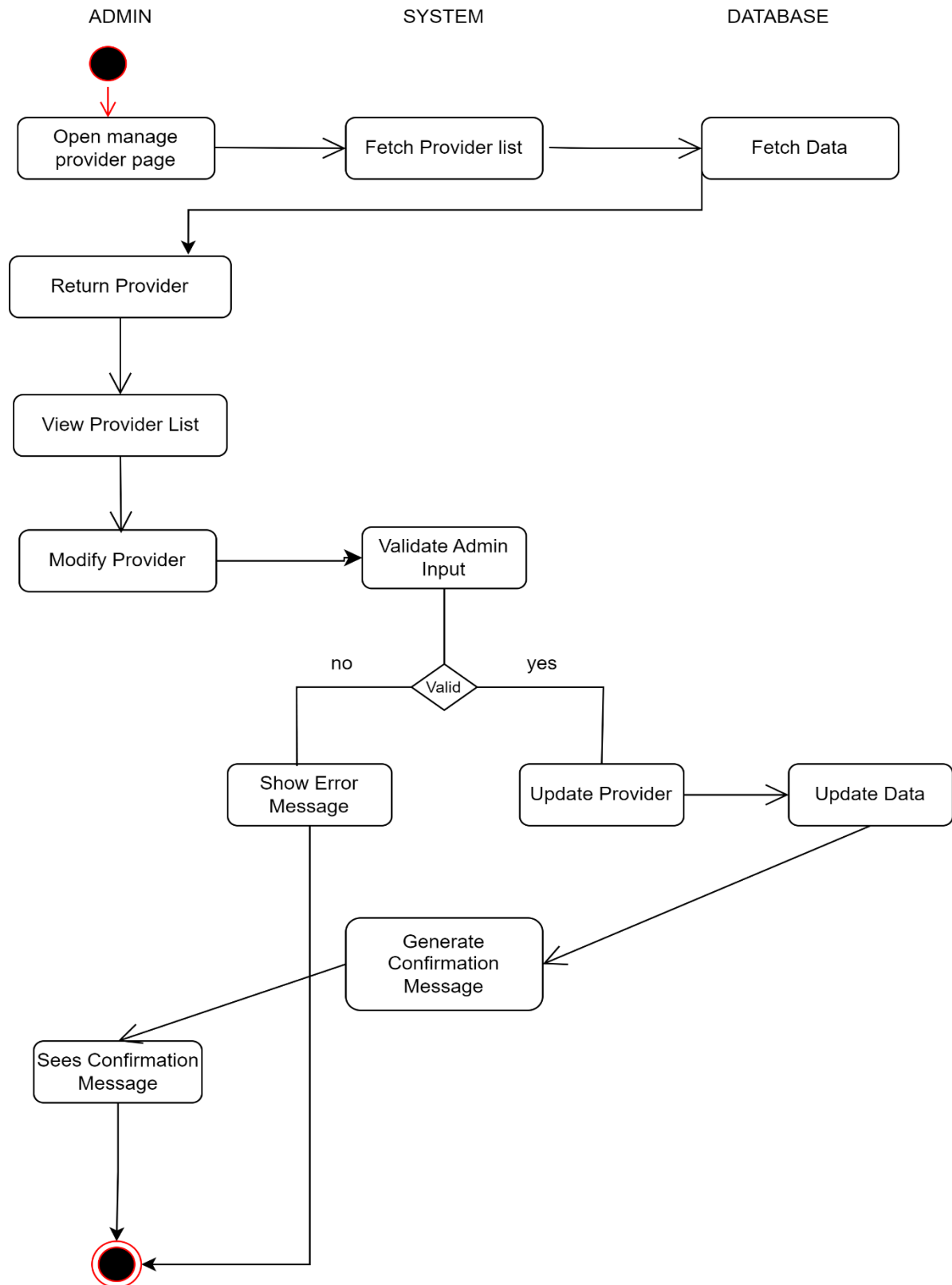


Figure 4.2:Activity Diagram: Admin Manage Provider

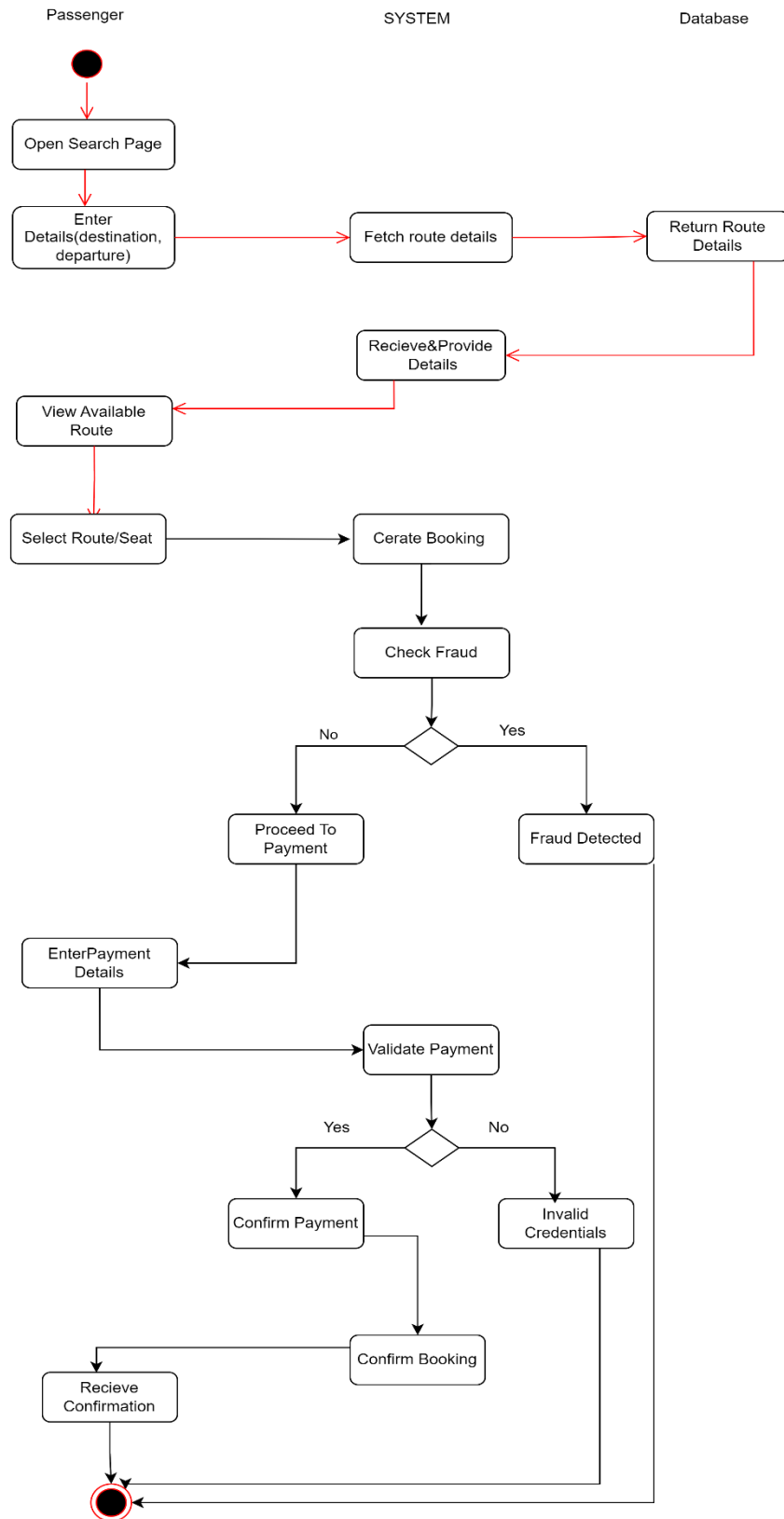


Figure 4.2:Activity Diagram: Booking Ticket

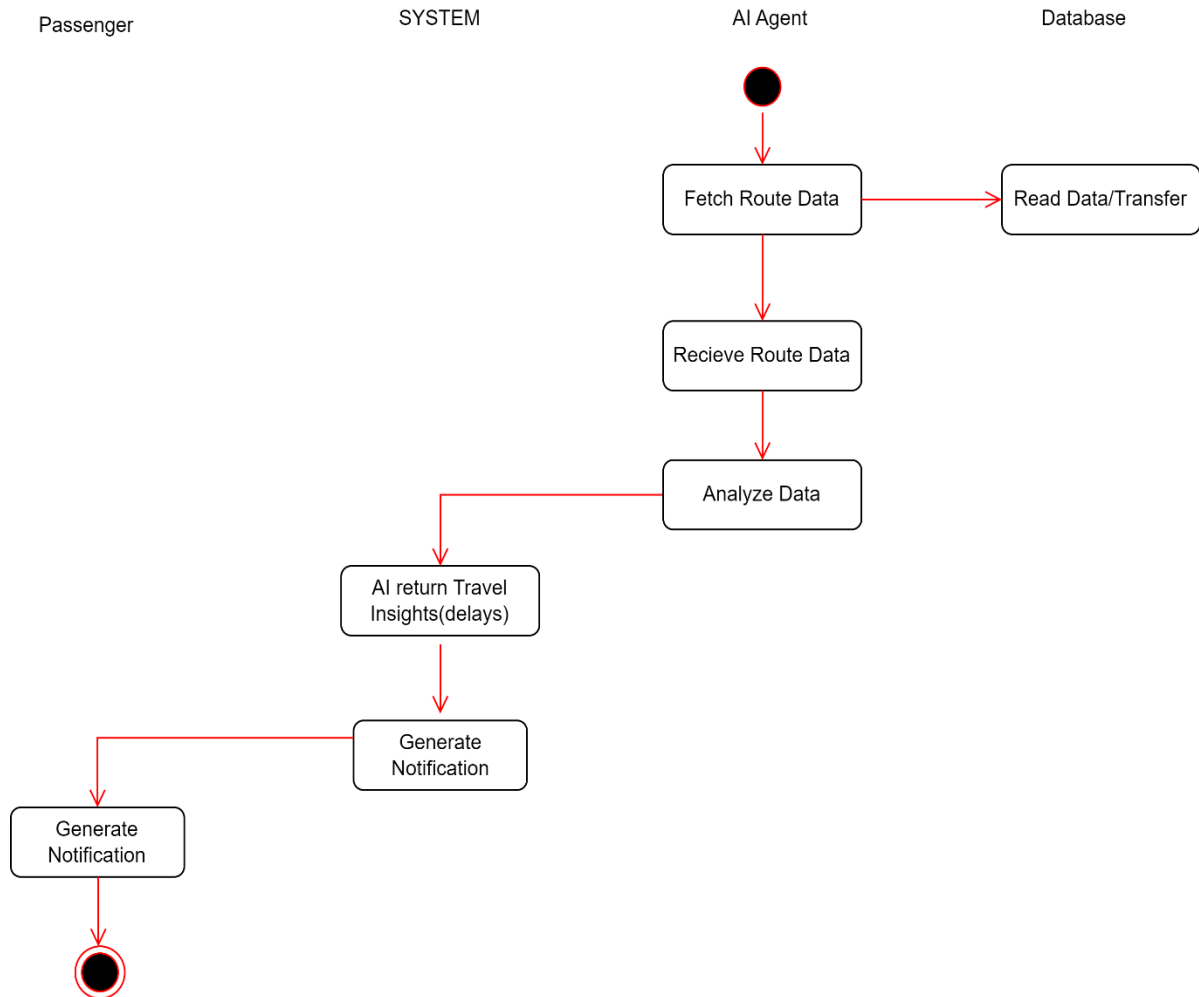


Figure 4.2:Activity Diagram: Real Time Travel Insights

4.3 UML Interaction Diagram

A UML Interaction Diagram is used to show how different objects or components communicate with each other within a system to accomplish a specific task. It focuses on the exchange of messages between system elements and helps explain the dynamic behavior of the system at runtime. Interaction diagrams describe the order, flow, and type of interactions that occur during processes such as login, data processing, analysis, or result generation.

4.3.1 Sequence Diagram

Sequence Diagram is a UML diagram that shows how different objects or system parts interact with each other by sending messages in a step-by-step, time-ordered sequence. It explains the step-by-step flow of messages between the user, system components, and database in the order they occur. The diagram is drawn from top to bottom, where time flows downward, making it easy to understand the sequence of actions.

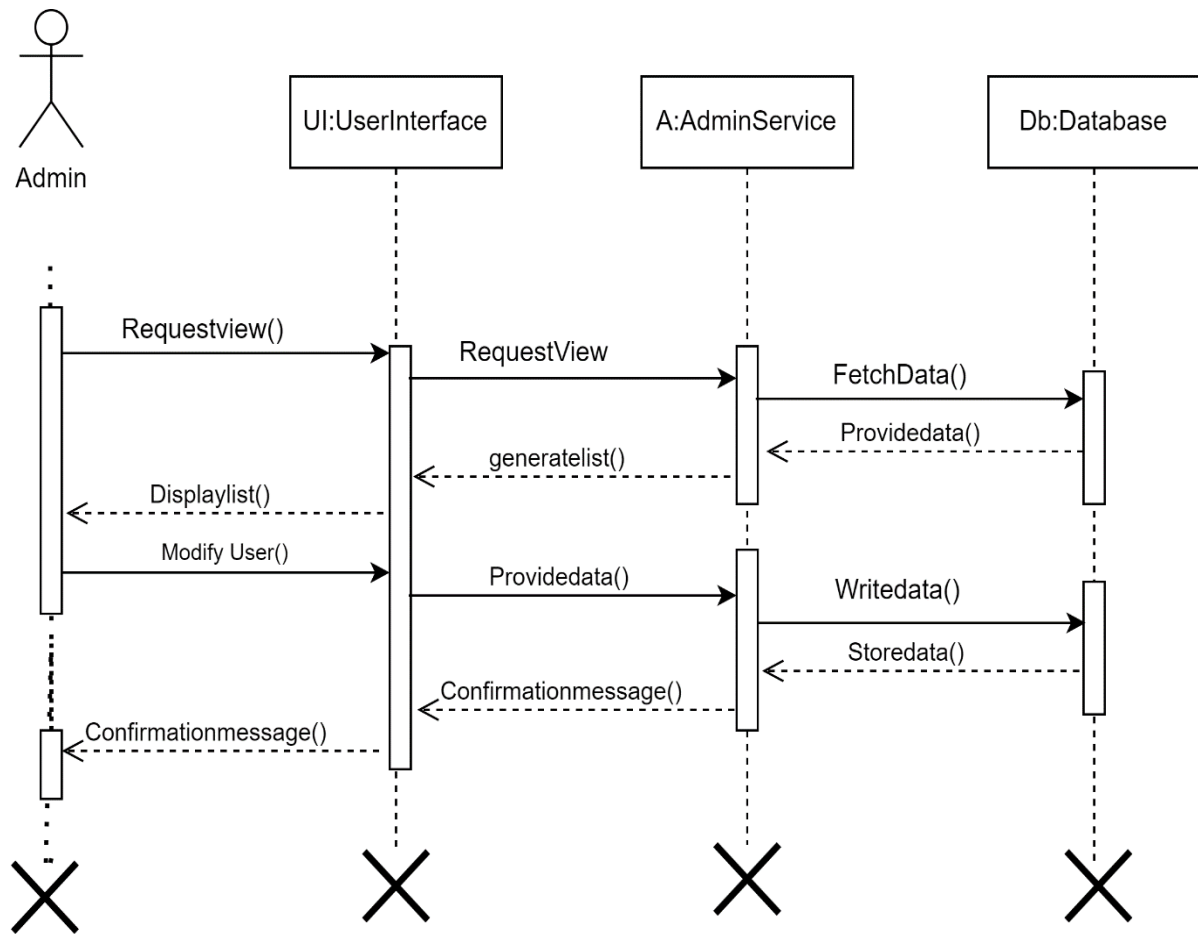


Figure 4.3:Sequence Diagram: Admin Manage User

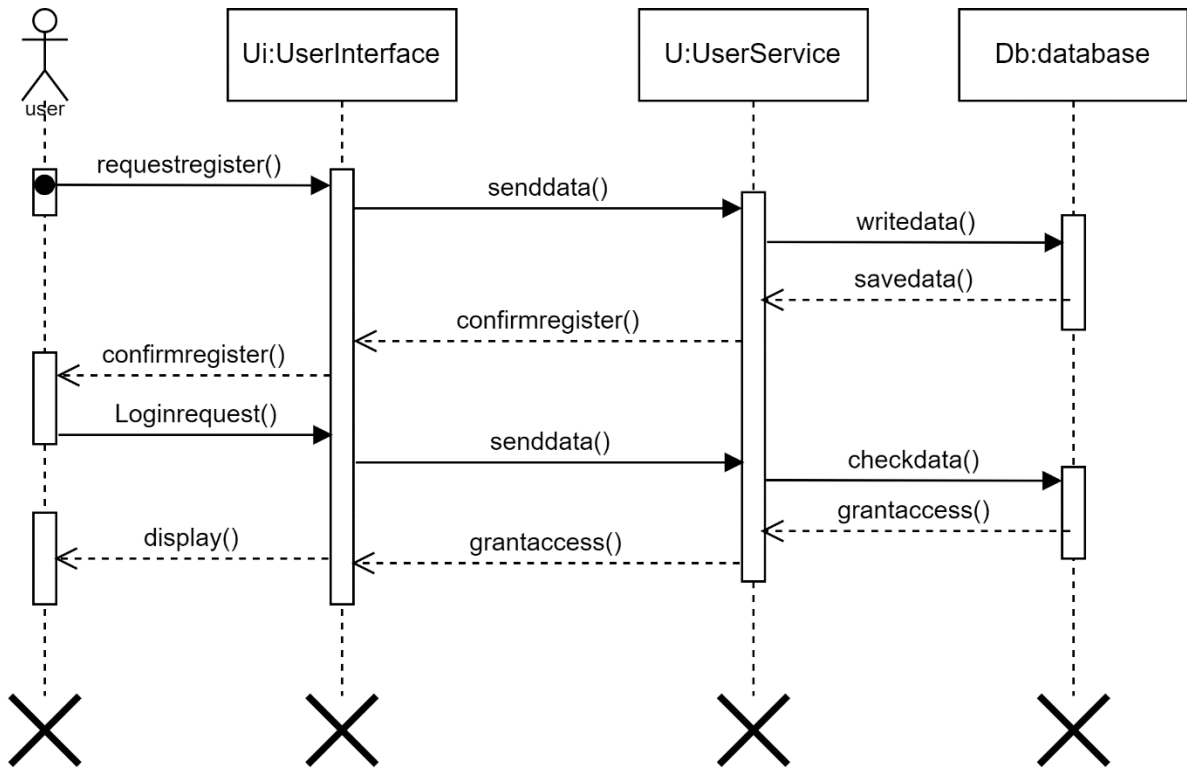


Figure 4.3:Sequence Diagram: User Registration/Login

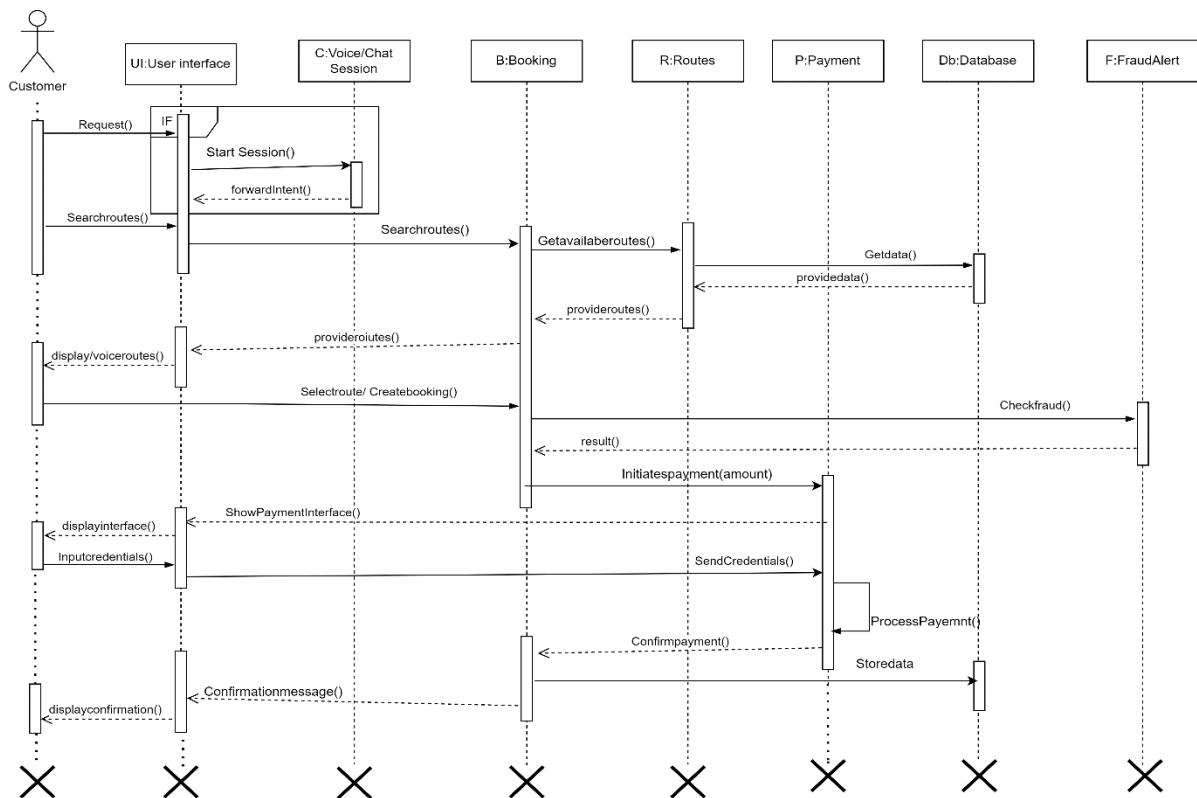


Figure 4.3:Sequence Diagram: Booking Ticket

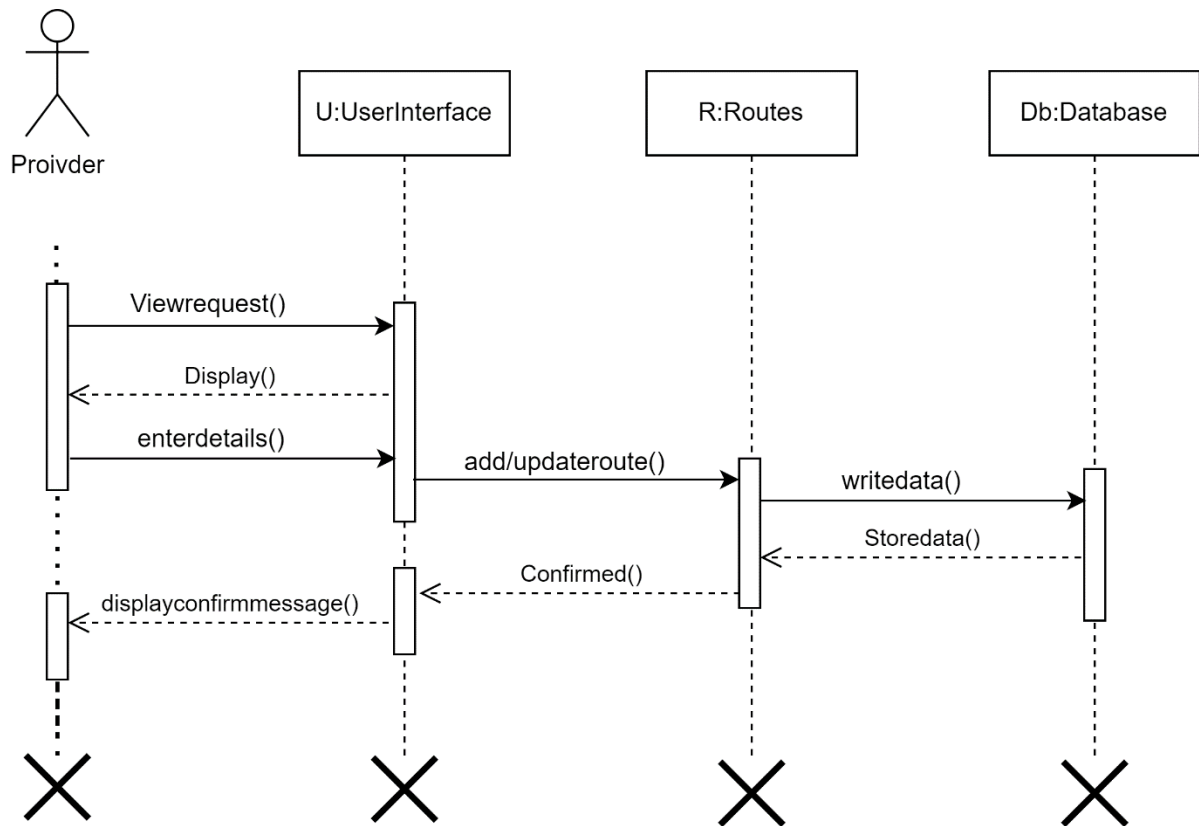


Figure 4.3:Sequence Diagram: Provider Manage Routes

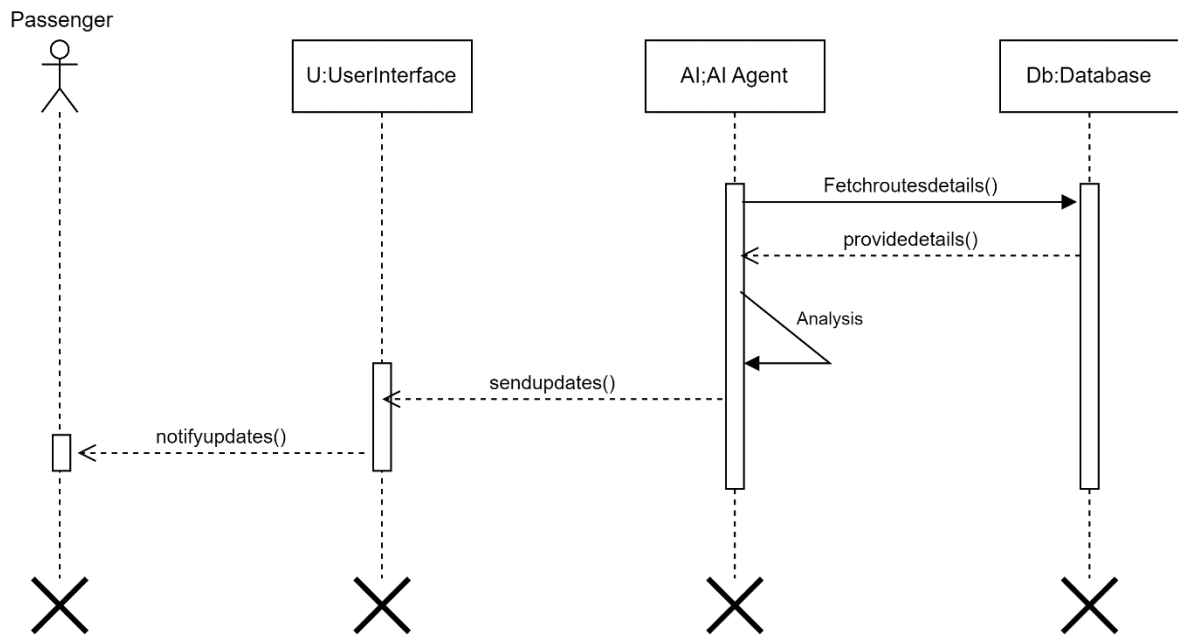


Figure 4.3:Sequence Diagram: Real Time Travel Updates

4.3.2 Collaboration Diagram

A collaboration diagram illustrates objects, their relationships, and the sequence of messages exchanged between them to achieve functionality.

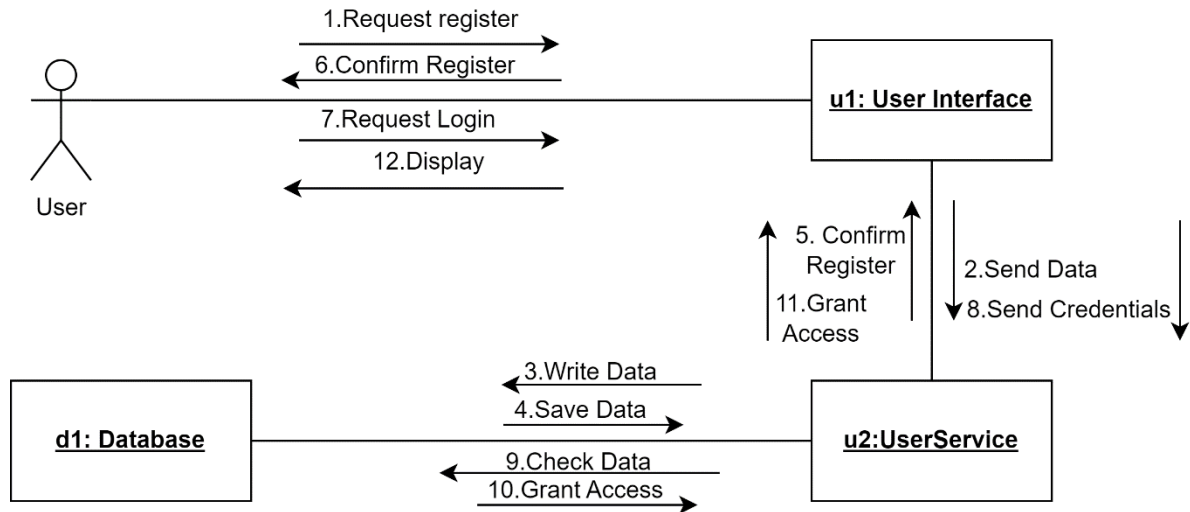


Figure 4.3:Collaboration Diagram: User registration/login

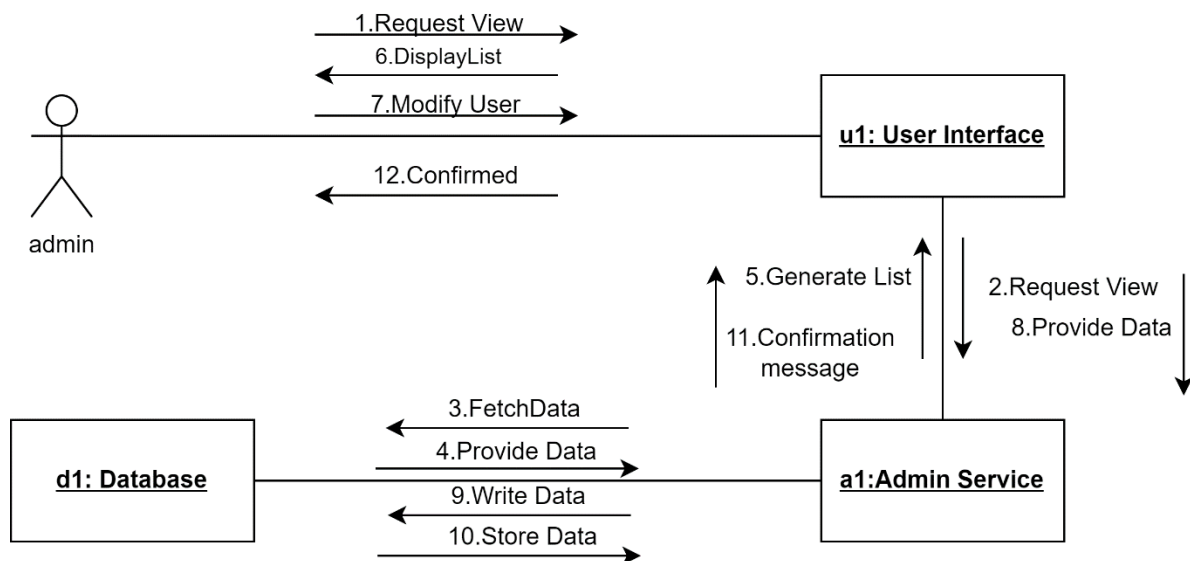


Figure 4.3:Collaboration Diagram: Admin Manage User

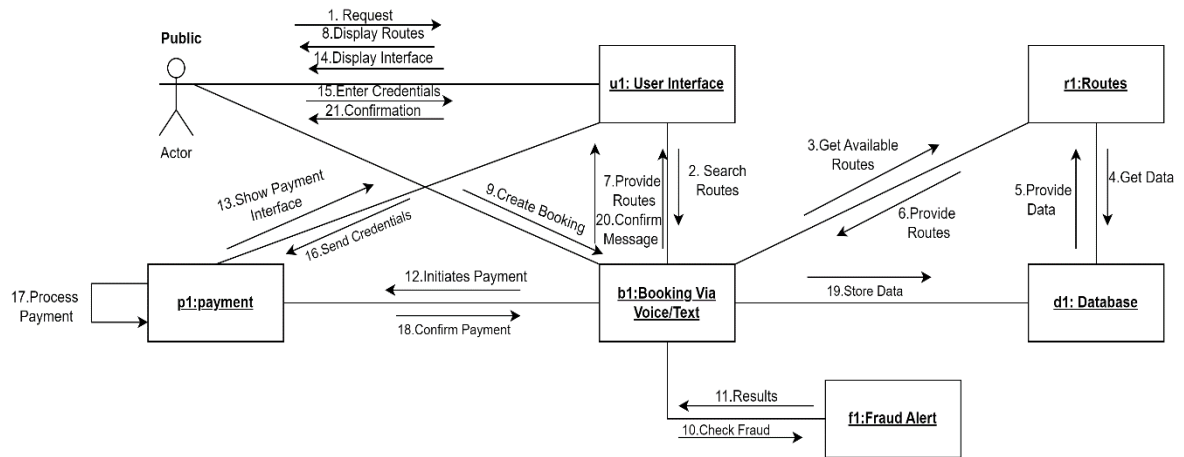


Figure 4.3: Collaboration Diagram: Booking Ticket

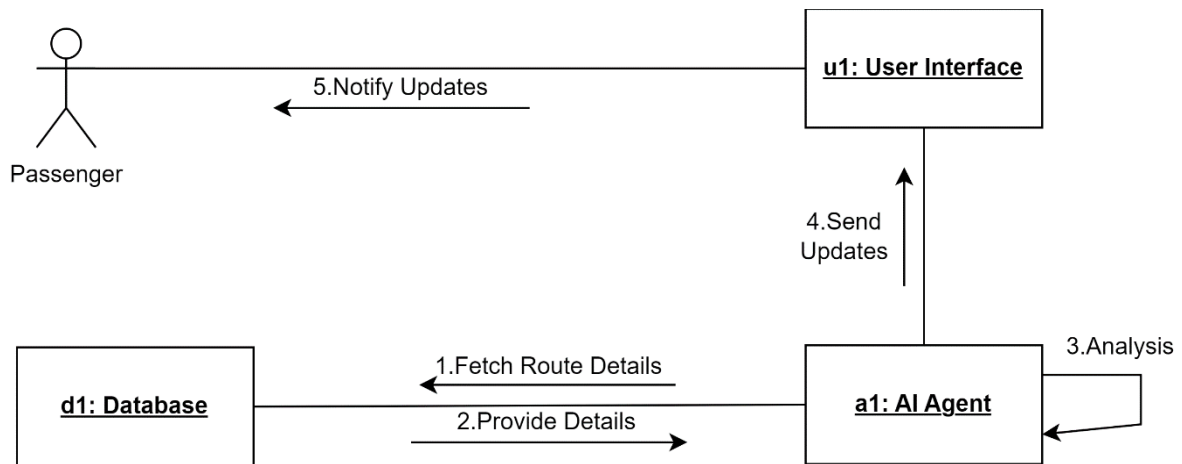


Figure 4.3: Collaboration Diagram: Real Time Travel Updates

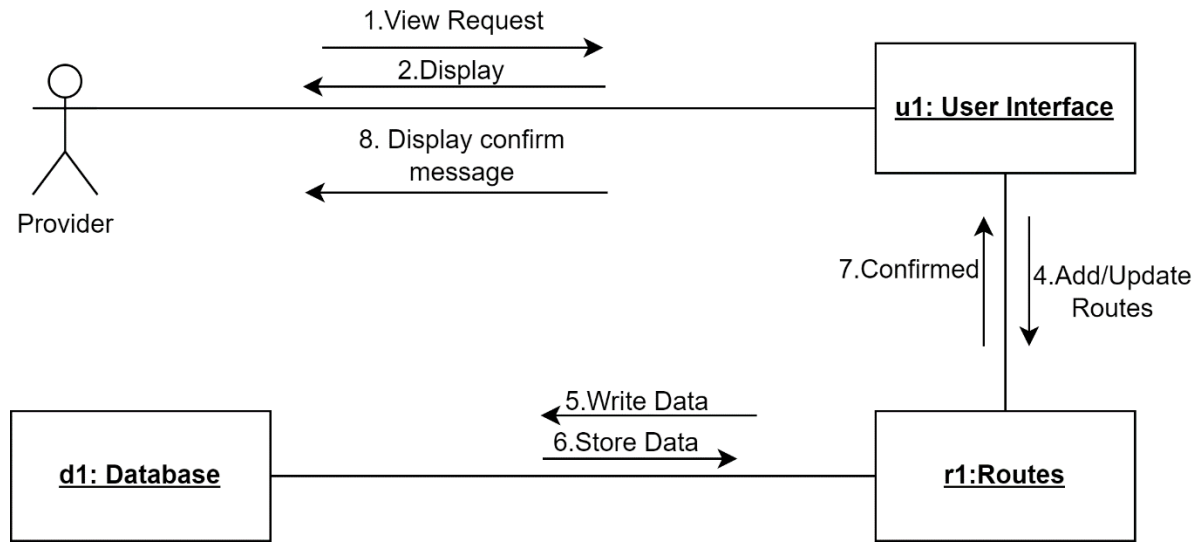


Figure 4.3: Collaboration Diagram: Provider Manage Routes

CHAPTER 5: IMPLEMENTATION

This chapter explains how the SafarBot system was practically developed and implemented. It describes the technologies used, system modules, communication methods, and how different components work together. The main goal of implementation was to build a simple, secure, and intelligent ticket booking system that can be easily used by all users, including uneducated and elderly people.

5.1 Communication Protocol Implementation

In the SafarBot web application, communication protocols define how different system components exchange data with each other. The React.js frontend communicates with the backend server using HTTP and HTTPS through RESTful APIs. All requests and responses are transferred securely in JSON format to ensure clarity, compatibility, and efficient data handling.

The backend server, developed using Node.js and Express.js, interacts with the MongoDB database using its native database protocol for storing and retrieving user data, bookings, routes, and fraud-related information. External services such as maps, weather, voice processing, and Stripe test mode APIs are accessed using HTTP-based APIs.

These communication protocols ensure secure, reliable, and smooth interaction between all SafarBot components, even during high user activity.

5.1.1 *Component Diagram*

The component diagram illustrates the interaction between the SafarBot frontend, backend services, database, AI modules, and external APIs. It shows how user requests flow from the UI to backend services and how responses are returned after processing. The diagram provides a clear view of how different system components are connected and communicate with each other. It also helps in understanding the overall system architecture and the role of each module in ensuring smooth and efficient operation.

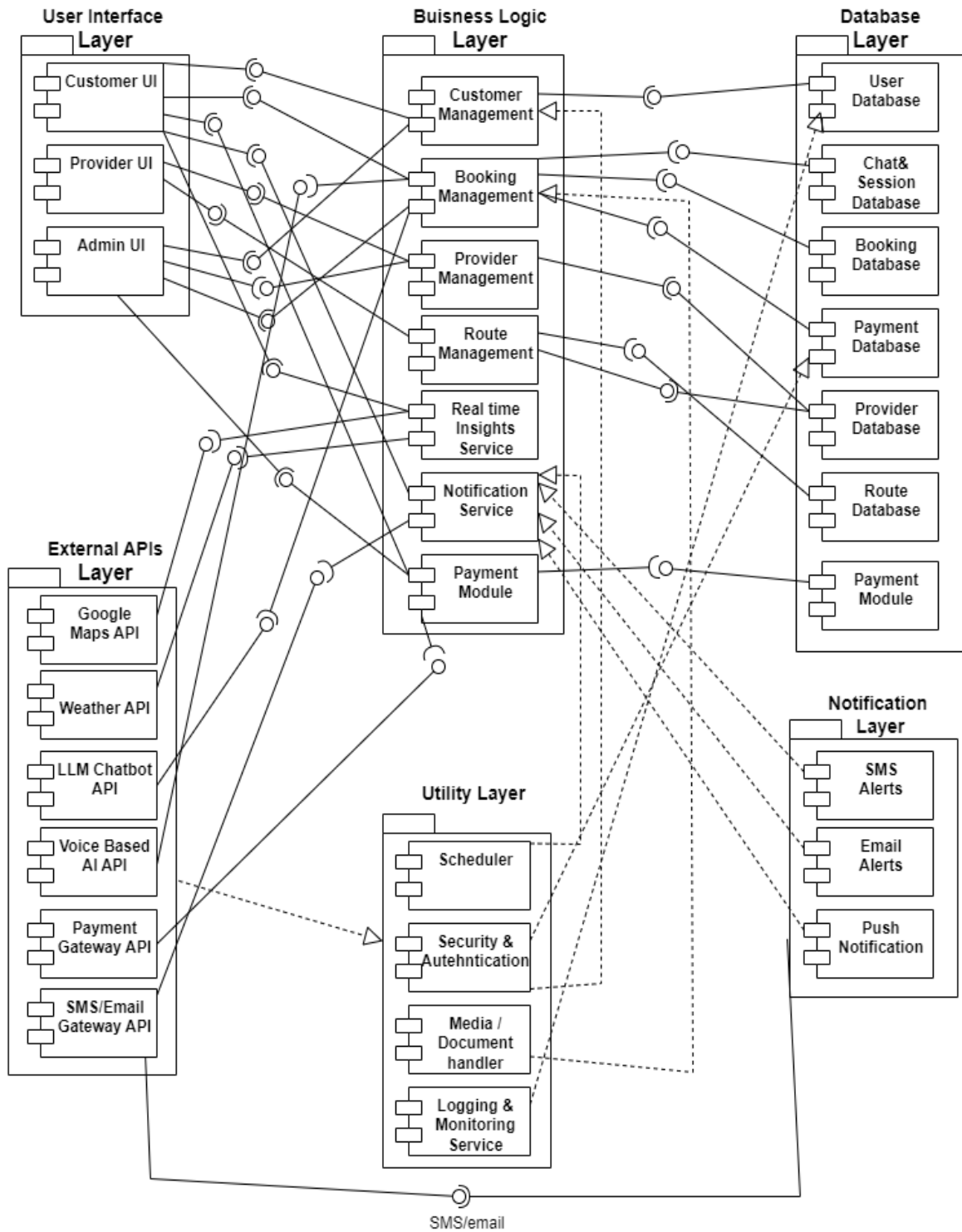


Figure 5.1:Component Diagram

5.2 Network and Protocol Choice

This section explains the communication technologies used to connect the frontend, backend, database, AI services, and external APIs. These protocol choices help ensure secure, reliable, and efficient data exchange across the SafarBot system.

5.2.1 *Communication Protocols*

These communication protocols ensure efficient, secure, and reliable data exchange between all components of the SafarBot system. They play a vital role in maintaining system performance, scalability, and seamless user interaction.

- ***HTTP/HTTPS Protocol***

The Hypertext Transfer Protocol (HTTP) and its secure version HTTPS are used as the core communication protocols between the client and server components of SafarBot. These protocols follow a request–response model, where the client sends a request and the server responds with the required data.

HTTP/HTTPS is lightweight, platform-independent, and widely supported, making it suitable for web-based ticket booking systems accessed from both desktop and mobile devices.

- ***RESTful APIs***

RESTful APIs are used to establish communication between the frontend, backend, database, AI services, and external systems. Standard HTTP methods such as GET, POST, PUT, and DELETE are used for retrieving, submitting, updating, and deleting data.

Data is exchanged in JSON format, which improves readability, interoperability, and scalability of the SafarBot system.

- ***Integration with External Services***

SafarBot integrates external services such as Google Maps, weather APIs, speech-to-text services, and Stripe test mode payment service using RESTful interfaces. These integrations enable real-time route visualization, travel updates, voice-based booking, and secure payment demonstrations without affecting system performance.

5.2.2 Security Considerations

Security considerations are essential to protect user data, ensure system integrity, and prevent unauthorized access. These measures help build user trust and maintain the reliability of the SafarBot platform.

- ***Authentication and Authorization***

Authentication mechanisms ensure that only registered and valid users can access the system. Role-based authorization is implemented to differentiate access levels for passengers, transport providers, and administrators. This prevents unauthorized access to sensitive booking and user data.

- ***Data Security and Encryption***

All sensitive data transmitted over the network is protected using secure HTTPS communication. User passwords and confidential information are encrypted before being stored in the database. This ensures data confidentiality and protects against data breaches.

- ***API Security and Input Validation***

APIs are secured using authentication tokens to prevent unauthorized requests. Input validation techniques are applied to protect the system from malicious inputs such as injection attacks. These measures enhance overall system security and reliability.

5.2.3 Performance Optimization

Performance optimization ensures that the system runs efficiently, even under high user load and peak booking times. It helps maintain fast response times, system stability, and a smooth user experience.

- ***Efficient Data Processing***

The system uses optimized database queries to reduce response time and server load. Only required data is fetched and processed, which improves system speed during peak booking times. This approach minimizes unnecessary data transfer and ensures efficient use of system resources. As a result, the application delivers faster performance, enhances user experience, and maintains stability even during high traffic conditions.

- ***Caching and Load Management***

Caching techniques are applied for frequently accessed data such as routes and schedules. This reduces repeated database access and improves response time. Load management strategies ensure system stability during high user traffic.

- ***Scalability and Network Efficiency***

The SafarBot architecture supports scalability to handle an increasing number of users and bookings. Efficient API calls reduce network latency and bandwidth usage, ensuring consistent performance during peak usage periods.

5.3 Choice of Object Middleware

This section describes how objects and middleware support the internal working of SafarBot. It explains how different modules manage users, bookings, authentication, validation, and real-time travel insights.

5.3.1 User Management Module

The User Management Module is responsible for handling all user-related operations within the system, ensuring secure access and proper role management. It plays a key role in maintaining system integrity, user authentication, and smooth interaction between different types of users.

Objects Used:

User objects store information such as name, email, role, and login credentials. These objects manage different user types including passengers, transport providers, and administrators. Object-oriented design improves data organization and code reusability.

Middleware Used:

Authentication middleware verifies user identity before granting system access. Authorization middleware enforces role-based access control. This separation of logic improves security and maintainability. It also helps in scaling the system efficiently by allowing independent updates to authentication and authorization logic without affecting core application features.

5.3.2 Ticket Booking Module

The Ticket Booking Module is responsible for managing all booking-related operations within the system, ensuring accurate ticket reservations, secure transactions, and proper seat allocation. It plays a key role in maintaining booking integrity, real-time availability updates, and smooth coordination between users and transport services.

Objects Used:

Booking objects store ticket details such as route, seat number, travel date, and payment status. These objects represent real-world booking transactions and allow structured data handling.

Middleware Used:

Validation middleware checks the completeness and correctness of booking requests. Error-handling middleware manages invalid inputs and system exceptions, ensuring reliable booking operations.

5.3.3 Real Time Travel Insights Module

The Real Time Travel Insights Module enhances the user experience by providing up-to-date travel information, enabling passengers to make informed decisions regarding routes, timing, and travel efficiency.

Objects Used:

Map objects are responsible for analyzing route-related data such as travel paths, traffic conditions, and pickup points. These objects generate processed travel insights and forward the results to passengers through the user interface.

Middleware Used:

Middleware handles the communication between map services and the system. It collects real-time map data, processes and formats the information, and ensures that the generated results are delivered accurately to the passenger interface.

5.4 User Interface

The ticket booking user interface is designed to be simple, clear, and user-friendly. Users can book tickets by selecting routes, seats, and travel dates through an easy-to-understand layout. Voice-based booking allows users to interact with the system without typing.

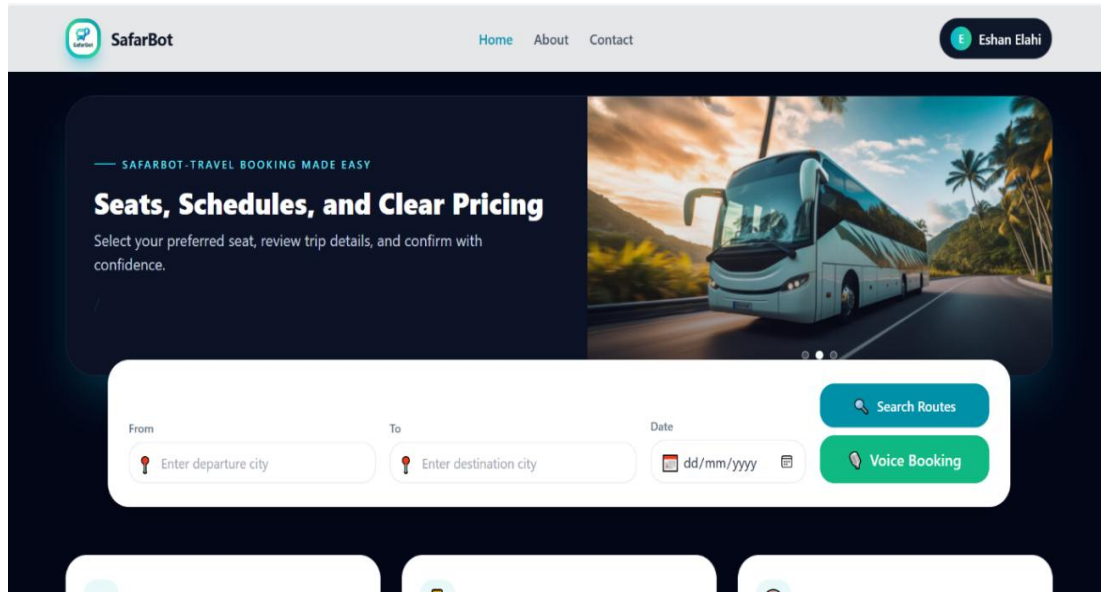


Figure 5.4:Home Page

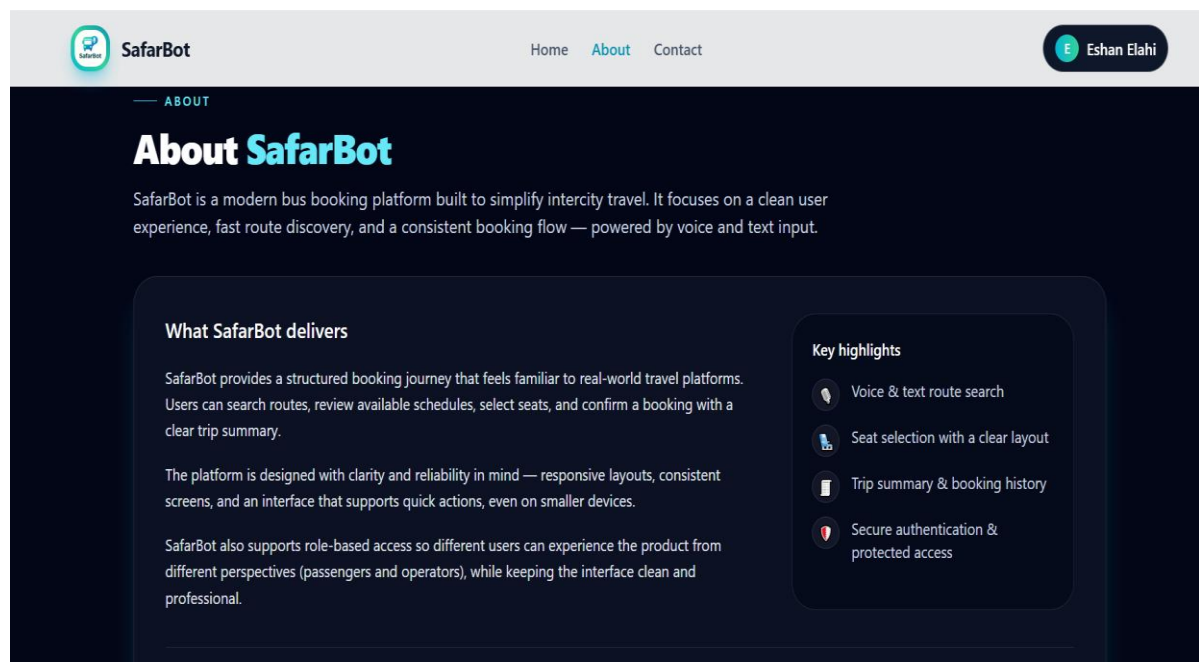


Figure 5.4:About Page

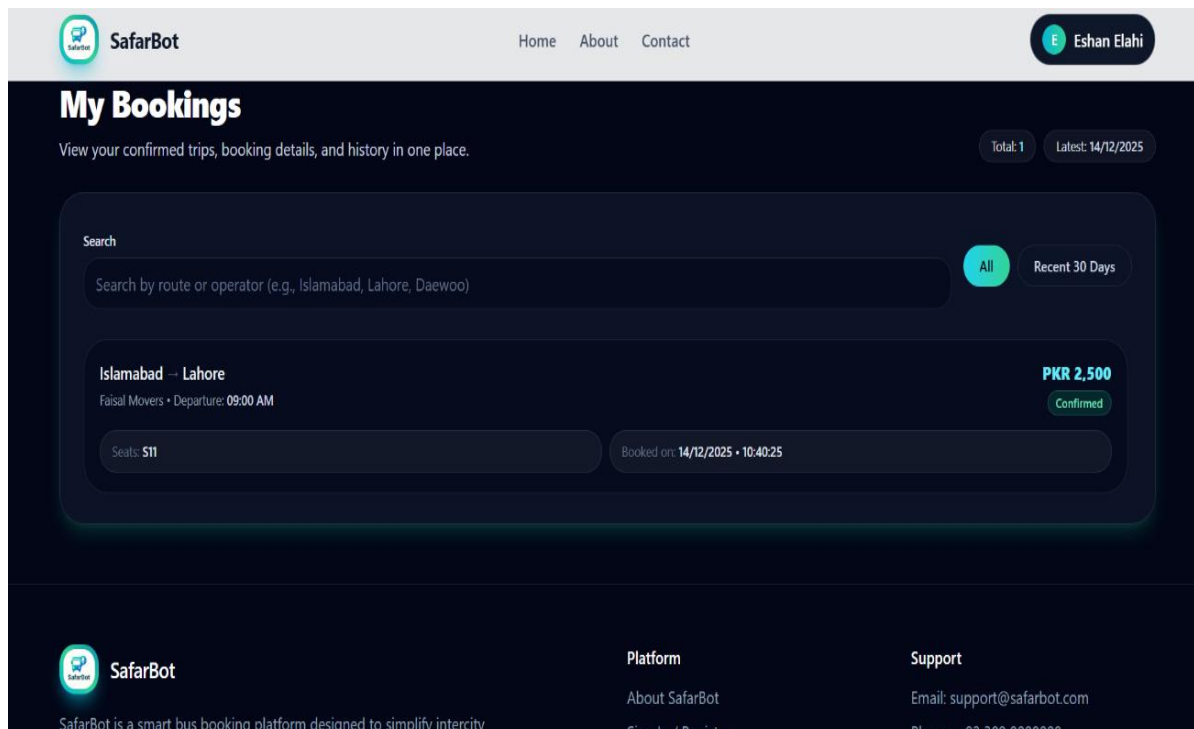


Figure 5.4:View Bookings

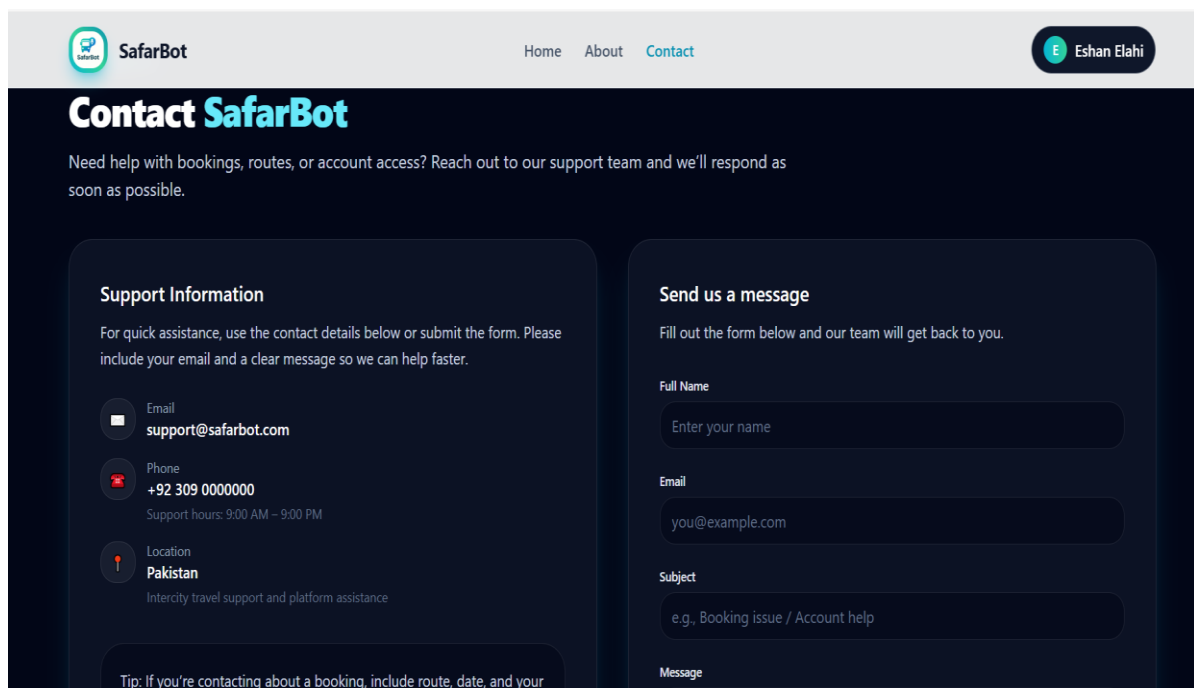


Figure 5.4:Contact Page

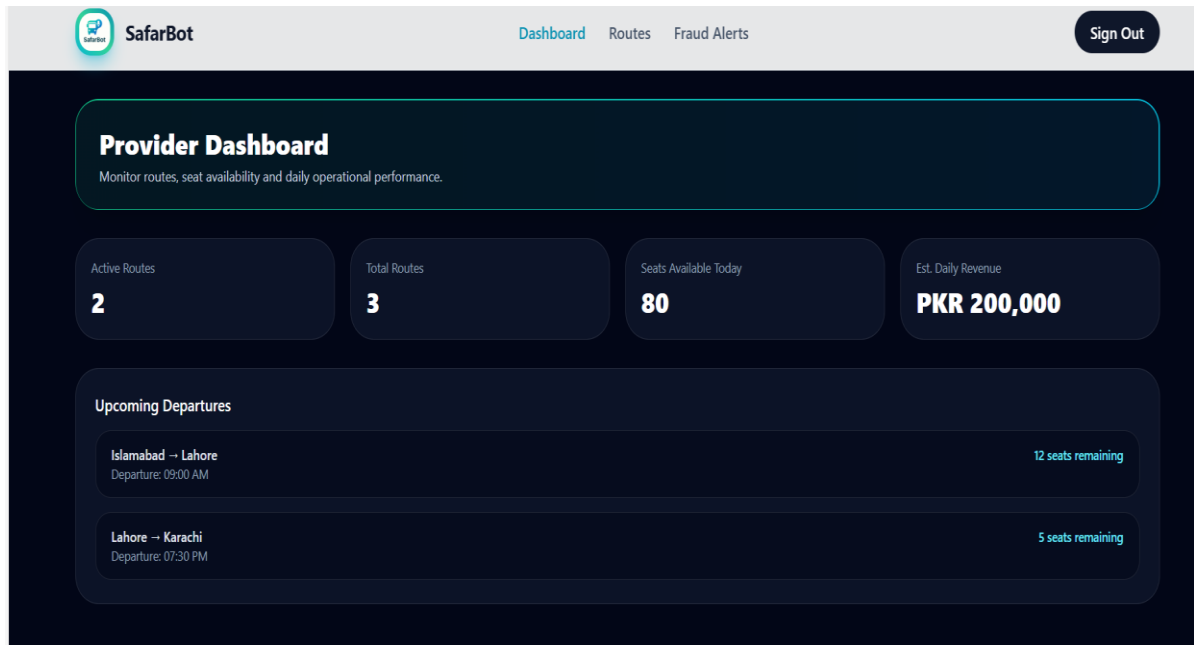


Figure 5.4:Provider Dashboard

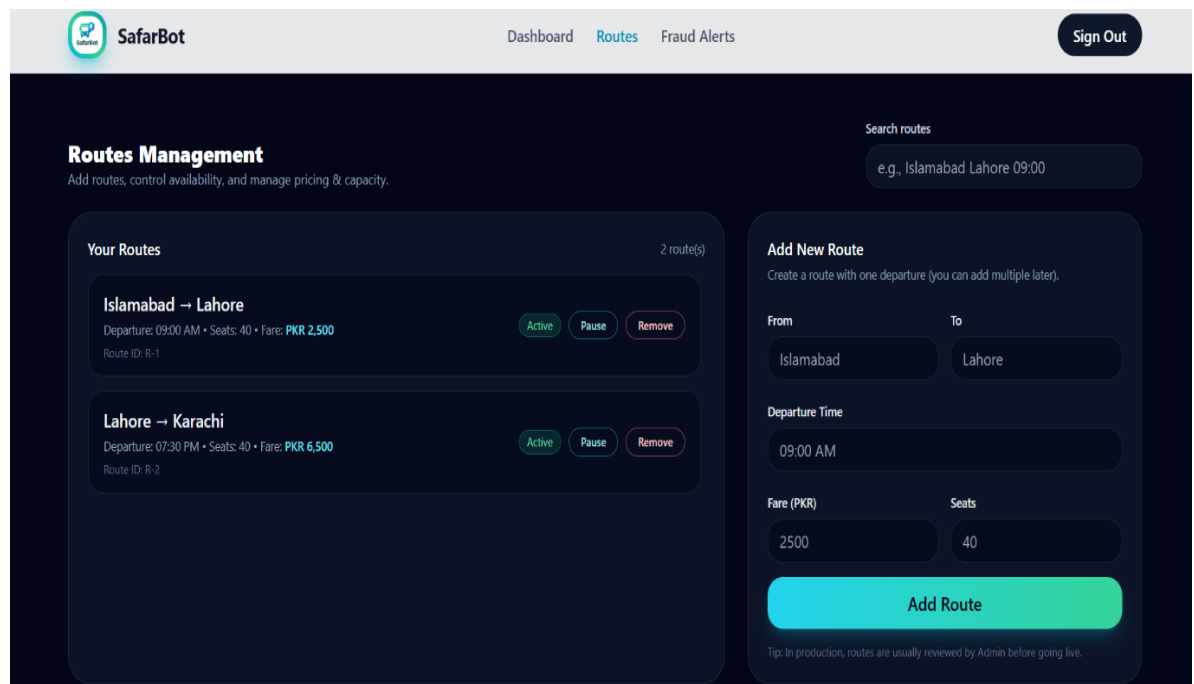


Figure 5.4:Routes Manage

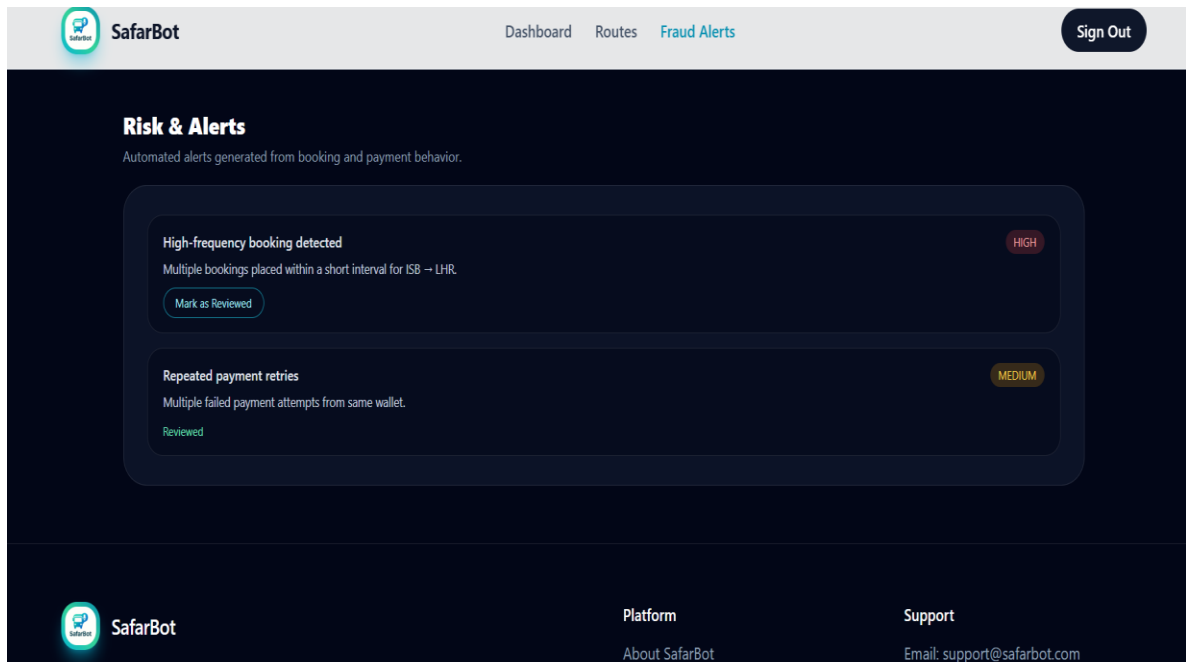


Figure 5.4: Fraud Alert

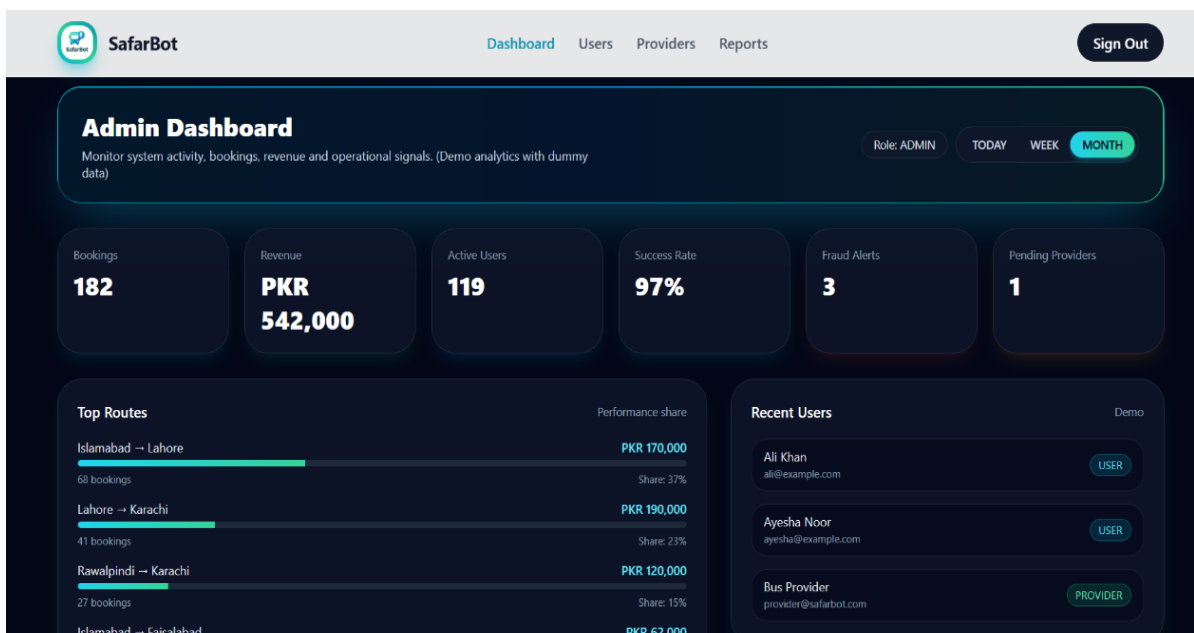


Figure 5.4: Admin Dashboard


SafarBot

[Dashboard](#)
[Users](#)
[Providers](#)
[Reports](#)

[Sign Out](#)

Manage Providers

Approve, suspend or review operators on the platform (demo data).

All Status ▼

Provider	Routes	Status	Actions
Faisal Movers	24	ACTIVE	View Edit Suspend
Daewoo Express	18	ACTIVE	View Edit Suspend
Skyways	6	PENDING	View Approve Suspend
Baloch Transport	10	SUSPENDED	View Edit Suspend


SafarBot

Platform

About SafarBot

Support

Email: support@safarbot.com

SafarBot is a smart bus booking platform designed to simplify intercity

Figure 5.4:Manage Provider


SafarBot

[Dashboard](#)
[Users](#)
[Providers](#)
[Reports](#)

[Sign Out](#)

Manage Users

Search accounts, filter roles and review status (demo data).

All Roles ▼

Name	Email	Role	Status	Actions
Ali Raza	ali@example.com	USER	ACTIVE	View Edit Disable
Fatima Khan	fatima@example.com	USER	ACTIVE	View Edit Disable
SafarBot Provider	provider@safarbot.com	PROVIDER	PENDING	View Edit Disable
SafarBot Admin	admin@safarbot.com	ADMIN	ACTIVE	View Edit Disable


SafarBot

Platform

About SafarBot

[Sign In / Register](#)

Support

Email: support@safarbot.com

Phone: +92 309 0000000

SafarBot is a smart bus booking platform designed to simplify intercity

Figure 5.4:Manage User

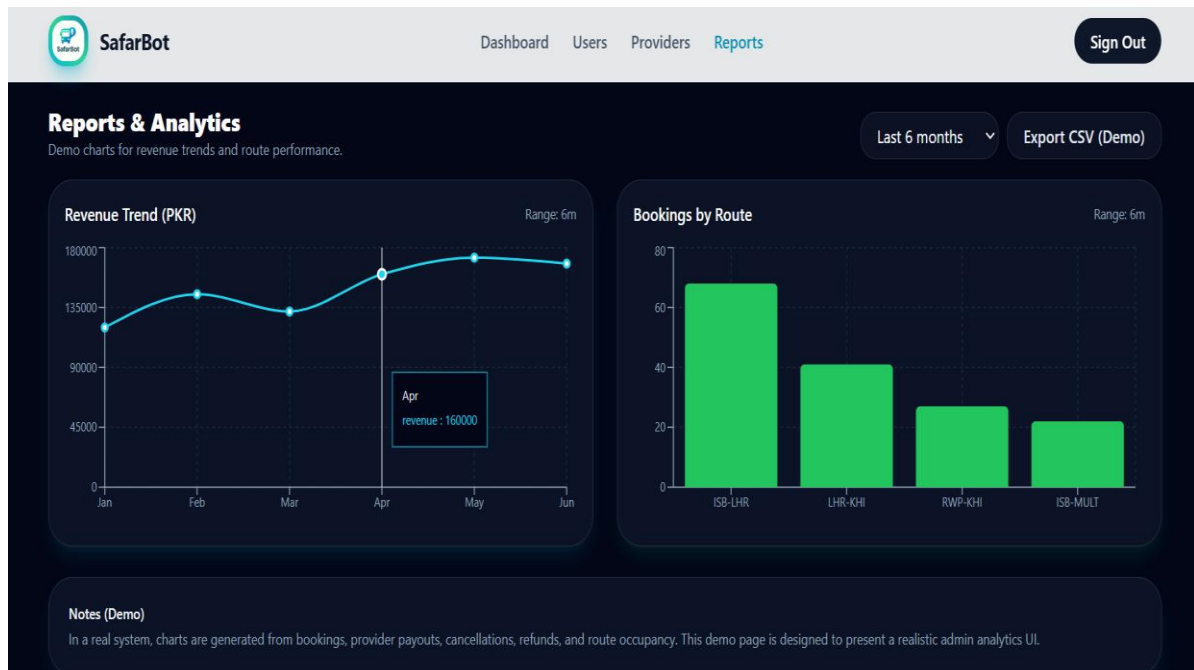


Figure 5.4:View Reports

The registration form is titled 'Create your SafarBot account' and includes fields for 'Full Name', 'Email', and 'Password'. A 'Create Account' button is at the bottom, and a 'Login' link is provided for existing users.

Create your SafarBot account
Create an account to book seats faster and manage your trips.

Full Name
Enter your full name

Email

Password
Minimum 6 characters

Create Account

Already registered? [Login](#)

Figure 5.4:Register Page



Welcome back to SafarBot
Sign in to continue your booking experience.

Email

Password

Minimum 6 characters

[Forgot password?](#)

Sign In

Don't have an account? [Register](#)

Figure 5.4:Login Page

CHAPTER 6: TESTING AND EVALUATION

This chapter discusses the testing and evaluation of the SafarBot system. Testing was performed to ensure that all system features work correctly, meet requirements, and provide a smooth user experience. Different testing techniques were applied at various stages of development.

6.1 Verification

Verification ensures that the system is built correctly according to the approved requirements, design specifications, and proposal.

6.1.1 *User Module Verification*

- User registration and login features match the project proposal.
- Authentication and authorization rules are properly implemented.
- No required user functionality is missing.

6.1.2 *Ticket Booking & Route Module Verification*

- Ticket booking, route selection, and seat management meet system specifications.
- Data flow between booking, payment, and database modules is correct.
- Design and functional requirements are fully fulfilled.

6.1.3 *Provider & Notification Module Verification*

- Transport provider features follow the approved system design.
- Notifications behave according to documented workflows.
- Security standards are properly implemented.

6.1.4 *AI Module Verification*

- AI-based features such as fraud detection and chat support meet defined requirements.
- Decision logic follows the designed AI model.
- Outputs are consistent and reliable.

6.2 Validation

Validation ensures that the system fulfills user needs and real-world usage expectations.

6.2.1 *User Validation*

- The system satisfies passenger and provider needs.
- Real-world booking scenarios work successfully.
- Users trust the system for ticket booking and payments.

6.2.2 *Functional Validation*

- All system modules work together correctly.
- The system performs according to expectations.
- Project objectives are successfully achieved.

6.2.3 *Performance Validation*

- The system performs efficiently under normal and high usage.
- Response time remains acceptable during peak booking hours.
- System reliability is confirmed through repeated testing.

6.3 Usability Testing

Usability testing ensures ease of use and accessibility for all users.

6.3.1 *User Interface Usability*

- The interface is easy to understand and navigate.
- Buttons, icons, and instructions are clear.
- Users can operate the system without prior training.

6.3.2 *Ticket Booking Usability*

- Booking steps are simple and straightforward.
- Minimum user input is required.
- Users can complete ticket bookings easily.

6.3.3 *Route & Notification Usability*

- Routes and schedules are clearly displayed.
- Notifications are noticeable and understandable.
- Users quickly grasp important travel information.

6.3.4 *Provider Dashboard Usability*

- Dashboard layout is clean and user-friendly.
- Booking and route management tasks are clearly shown.
- Navigation between features is smooth.

6.4 Unit Testing

Unit testing was performed to verify the individual modules of SafarBot independently before integration. The following test cases focus on core functions that can be tested with real input values, expected results, and actual outcomes. The previous step-based format has been replaced with an Input field to make each unit test more precise and executable.

Table 6.1: User Registration with Valid Details

Field	Details
Test Case ID	TC-01
Test Case Name	User Registration with Valid Details
Description	Verify that a new passenger account is created when valid registration data is provided.
Prerequisite	The user email must not already exist in the database, and the backend server must be running.
Input	{ "name": "Ali Khan", "email": "ali.khan@safarbot.pk", "password": "Ali@12345", "role": "Passenger" }

Expected Result	The system creates a new passenger account, stores the password securely, and returns a successful registration response.
Result	User account created successfully.
Pass/Fail	Pass

Table 6.2: User Login with Valid Credentials

Field	Details
Test Case ID	TC-02
Test Case Name	User Login with Valid Credentials
Description	Verify that an existing user can log in successfully using correct email and password.
Prerequisite	A registered user account must exist in the database.
Input	{ "email": "ali.khan@safarbot.pk", "password": "Ali@12345" }
Expected Result	The system validates the credentials and returns a valid authentication token for the user session.
Result	Login completed and authentication token generated.
Pass/Fail	Pass

Table 6.3: Route Search with Valid Details

Field	Details
Test Case ID	TC-03
Test Case Name	Route Search with Valid Details

Description	Verify that the route search function returns available routes for a valid source, destination, and travel date.
Prerequisite	At least one route from Rawalpindi to Lahore must exist in the route collection.
Input	{ "source": "Rawalpindi", "destination": "Lahore", "travelDate": "2026-06-15" }
Expected Result	The system returns matching routes with schedule, fare, provider, and seat availability details.
Result	Matching route list displayed successfully.
Pass/Fail	Pass

Table 6.4: Seat Availability Validation

Field	Details
Test Case ID	TC-04
Test Case Name	Seat Availability Validation
Description	Verify that the system correctly checks whether a selected seat is available before booking.
Prerequisite	The selected route and travel date must exist in the system.
Input	{ "routeId": "RWP-LHR-101", "seatNumber": "A3", "travelDate": "2026-06-15" }
Expected Result	The system verifies the selected seat status and allows selection only if the seat is available.

Result	Seat availability verified successfully.
Pass/Fail	Pass

Table 6.5: Ticket Booking with Valid Data

Field	Details
Test Case ID	TC-05
Test Case Name	Ticket Booking with Valid Data
Description	Verify that the ticket booking function creates a booking record when valid booking information is submitted.
Prerequisite	The user must be authenticated, the route must exist, and the selected seat must be available.
Input	{ "userId": "USR-1001", "routeId": "RWP-LHR-101", "seatNumber": "A3", "travelDate": "2026-06-15", "passengerName": "Ali Khan" }
Expected Result	The system creates a booking record with correct user, route, seat, travel date, and initial payment status.
Result	Booking record created successfully.
Pass/Fail	Pass

Table 6.6: Stripe Test Payment Success

Field	Details
Test Case ID	TC-06
Test Case Name	Stripe Test Payment Success

Description	Verify that the payment function processes a successful transaction using valid Stripe test card details.
Prerequisite	A pending booking must exist, and Stripe test mode must be configured.
Input	{ "bookingId": "BK-5001", "amount": 2500, "currency": "PKR", "card": "4242 4242 4242 4242", "expiry": "12/30", "cvc": "123" }
Expected Result	The system marks the payment as successful and updates the booking status to confirmed.
Result	Payment processed and booking confirmed successfully.
Pass/Fail	Pass

Table 6.7: Voice Booking Input Processing

Field	Details
Test Case ID	TC-07
Test Case Name	Voice Booking Input Processing
Description	Verify that the voice-processing module extracts booking details from a valid voice command.
Prerequisite	Microphone permission and speech-to-text service must be available in the test environment.
Input	Voice/Text Input: "Book a ticket from Rawalpindi to Lahore for 15 June 2026"
Expected Result	The system extracts source as Rawalpindi, destination as Lahore, and travel date as 2026-06-15.

Result	Voice command processed and booking details extracted.
Pass/Fail	Pass

Table 6.8: AI Fraud Detection for Duplicate Booking

Field	Details
Test Case ID	TC-08
Test Case Name	AI Fraud Detection for Duplicate Booking
Description	Verify that the fraud detection function identifies a duplicate booking attempt for the same user, route, seat, and date.
Prerequisite	An existing booking with the same user, route, seat, and travel date must already be stored.
Input	{ "userId": "USR-1001", "routeId": "RWP-LHR-101", "seatNumber": "A3", "travelDate": "2026-06-15", "paymentStatus": "Pending" }
Expected Result	The AI module assigns a high-risk score and generates a duplicate booking alert for review.
Result	Duplicate booking risk detected successfully.
Pass/Fail	Pass

Table 6.9: Real-Time Travel Insight Retrieval

Field	Details
Test Case ID	TC-09

Test Case Name	Real-Time Travel Insight Retrieval
Description	Verify that the travel insight module returns route-related weather, traffic, and delay information for valid route data.
Prerequisite	External map/weather APIs or configured test responses must be available.
Input	{ "route": "Rawalpindi-Lahore", "latitude": 33.5651, "longitude": 73.0169, "dateTime": "2026-06-15T08:00:00" }
Expected Result	The system returns current weather, traffic condition, and estimated delay information for the selected route.
Result	Real-time travel insights retrieved successfully.
Pass/Fail	Pass

Table 6.10: Provider Route Creation with Valid Data

Field	Details
Test Case ID	TC-10
Test Case Name	Provider Route Creation with Valid Data
Description	Verify that a provider can create a new route using complete and valid route information.
Prerequisite	The provider must be authenticated and approved by the administrator.
Input	{ "providerId": "PRV-2001", "source": "Rawalpindi", "destination": "Lahore", "departureTime": "08:00 AM", "fare": 2500, "totalSeats": 40 }

Expected Result	The system saves the new route and makes it available for user search and booking.
Result	Provider route created successfully.
Pass/Fail	Pass

Table 6.11: Booking Confirmation Notification Generation

Field	Details
Test Case ID	TC-11
Test Case Name	Booking Confirmation Notification Generation
Description	Verify that the notification module generates a booking confirmation alert after a successful booking.
Prerequisite	A confirmed booking must exist in the system.
Input	{ "userId": "USR-1001", "bookingId": "BK-5001", "eventType": "BOOKING_CONFIRMED" }
Expected Result	The system creates a notification containing booking confirmation details for the passenger.
Result	Booking confirmation notification generated successfully.
Pass/Fail	Pass

Table 6.12: Admin Fraud Alert View

Field	Details
Test Case ID	TC-12

Test Case Name	Admin Fraud Alert View
Description	Verify that the admin fraud-alert function retrieves high-risk fraud alerts for administrator review.
Prerequisite	The admin must be authenticated, and at least one high-risk alert must exist.
Input	{ "adminId": "ADM-001", "filter": "High Risk", "date": "2026-06-15" }
Expected Result	The system returns matching high-risk fraud alerts and prevents unauthorized users from viewing them.
Result	High-risk fraud alerts displayed successfully.
Pass/Fail	Pass

6.5 Integration Testing

Integration testing ensures smooth interaction between modules.

6.5.1 User Login & Ticket Booking Integration

- Only authenticated users can book tickets.
- Booking data flows correctly between modules.

6.5.2 Ticket Booking & Routes Integration

- Booked routes appear correctly on maps.
- Location data integrates properly with mapping services.
- Map updates occur without delay.

6.5.3 AI Fraud Detection & Booking Integration

- Fraud analysis integrates with the booking process.
- Suspicious activities are flagged correctly.

- System responses remain accurate.

6.5.4 *Provider Management & Booking Integration*

- Provider schedules integrate with user bookings.
- Seat availability updates correctly.
- Coordination remains consistent.

6.5.5 *Notification & Booking Integration*

- Notifications trigger after bookings or changes.
- Relevant users receive accurate messages.
- Delivery is reliable.

6.5.6 *Database & Backend Integration*

- All modules interact correctly with the database.
- Data consistency is maintained.
- No data loss occurs.

6.6 System Testing

Complete system testing validates end-to-end workflows.

6.6.1 *User Registration & Login Module*

- Users can register and log in successfully.
- Secure access is ensured for passengers, providers, and admins.

6.6.2 *Ticket Booking Module*

- Users can book tickets without errors.
- Booking details are stored accurately.
- Invalid inputs are handled properly.

6.6.3 *Route & Map Module*

- Routes and travel paths display correctly.

- Real-time updates work as expected.

6.6.4 *Provider Management Module*

- Providers can manage routes and schedules.
- Duplicate or incorrect entries are prevented.

6.6.5 *AI Module*

- AI features operate correctly.
- Errors and missing data are handled safely.

6.7 *Acceptance Testing*

Acceptance testing confirms system readiness for real-world use.

6.7.1 *User Module Acceptance*

- Users confirm easy registration and login.
- System usability is approved.

6.7.2 *Ticket Booking Acceptance*

- Users find the booking process simple.
- Tickets appear correctly after booking.

6.7.3 *Route Map Acceptance*

- Routes are clear and understandable.
- System is accepted for real use.

6.7.4 *Provider Module Acceptance*

- Providers confirm correct schedule and seat management.
- Response time is acceptable.

6.7.5 *AI Module Acceptance*

- AI outputs are understandable.
- Fraud alerts support decision-making.

6.8 Stress Testing

Stress testing checks how SafarBot performs under heavy load and unusual usage conditions. It helps verify whether the system can remain stable when many users, bookings, notifications, or AI requests occur at the same time.

6.8.1 High User Load Testing

- The system is tested with many simultaneous users.
- Performance remains stable during peak booking times.

6.8.2 Multiple Booking Requests

- Stress testing checks system behavior under heavy booking activity.
- Database handles large data volumes efficiently.

6.8.3 Notification Load Testing

- System sends notifications to a large number of users simultaneously.
- Delivery remains timely without failures.

6.8.4 AI Processing Load

- AI processing are tested under heavy load.
- Accuracy and response time remain acceptable.

6.9 Hardware Configuration for Testing

This section lists the hardware and network conditions used to test the system. It ensures that SafarBot is evaluated on machines and environments suitable for realistic development and usage scenarios.

6.9.1 Client-Side Hardware

- Intel Core i5 or above
- Minimum 8 GB RAM
- Stable internet connection

6.9.2 *Server-Side Hardware*

- Cloud server with minimum 8 GB RAM
- Multi-core processor
- SSD storage

6.9.3 *Network Requirements*

- High-speed internet for real-time operations
- Tested under low and high bandwidth conditions

6.10 Evaluation

The evaluation section summarizes the overall performance, usability, reliability, and functional success of SafarBot. It confirms whether the developed system meets the project objectives and user expectations.

6.10.1 *System Evaluation – Safarbot*

The evaluation of SafarBot is conducted to ensure that the system meets its objectives, performs reliably, and provides efficient ticket booking services.

1. *User Module Evaluation*

Users can register and log in easily. Secure role-based access is maintained.

2. *Ticket Booking Module Evaluation*

Bookings are completed successfully. Data accuracy and validation are confirmed.

3. *Route & Map Module Evaluation*

Routes are displayed clearly and updated in real time.

4. *Provider Module Evaluation*

Providers manage routes and schedules effectively.

5. *AI Module Evaluation*

Fraud detection improves booking security.

6. *Notification Module Evaluation*

Users receive timely and accurate notifications.

7. ***Database & Backend Evaluation***

Data storage is secure and reliable.

8. ***Performance Evaluation***

System remains responsive under heavy usage.

9. ***Usability Evaluation***

Users can operate the system without training.

10. ***Overall System Evaluation***

SafarBot successfully meets all project objectives.

6.11 Deployment

SafarBot is deployed in a cloud-based environment to ensure availability, scalability, and security.

- Frontend deployed on web hosting platform
- Backend deployed on secure cloud server
- Database hosted on cloud database service
- AI models deployed on Hugging Face and served through FastAPI
- Notification services configured

Security, monitoring, and final deployment verification ensure readiness for real-world use.

6.12 Maintenance

System maintenance ensures long-term reliability and performance.

- Corrective maintenance fixes errors
- Adaptive maintenance supports new features
- Preventive maintenance reduces failures
- Perfective maintenance improves usability
- Database and security maintenance ensure safety
- Hugging Face AI models are updated regularly

CHAPTER 7: CONCLUSION AND FUTURE WORK

This chapter concludes the SafarBot project by summarizing its achievements and discussing possible future improvements. It highlights how the system supports accessible, intelligent, and secure online ticket booking while providing a foundation for future enhancements.

7.1 Conclusion

The SafarBot project successfully achieves its objective of developing an AI-powered, voice-enabled transport ticket booking system tailored to the Pakistani context. The system addresses key limitations of existing platforms, such as lack of accessibility, absence of voice interaction, limited automation . By integrating bilingual voice-based booking (Urdu and English), real-time travel insights, and intelligent automation, SafarBot enhances usability for elderly, low-literacy, and non-English-speaking users.

The project utilizes modern technologies including the MERN stack, Python/FastAPI-based AI services with Hugging Face model deployment, RESTful APIs, JWT authentication, and cloud-based deployment to ensure scalability, security, and performance. The real-time travel insights module and voice booking system have been successfully implemented, along with complete frontend and backend integration. The modular architecture, Agile development methodology, object-oriented design principles, UML modeling, and structured testing approach demonstrate sound software engineering practices.

Overall, SafarBot contributes toward building an inclusive, secure, and intelligent digital booking ecosystem. The system improves user trust, enhances operational efficiency for transport providers, and lays the foundation for future AI-driven advancements in the transport industry.

7.2 Future Work

Although SafarBot has successfully implemented real-time insights, voice-based booking, frontend, and backend modules, several enhancements can be incorporated in future versions to further strengthen security, intelligence, and scalability.

1. Payment Gateway Integration

Future development will include full Stripe live mode integration for real transaction processing. Additional features may include secure refund handling and transaction tracking mechanisms.

2. Advanced AI-Based Fraud Detection

The fraud detection module will be further developed using machine learning models trained on real booking datasets. Future enhancements may include:

- Behavioral anomaly detection based on booking patterns
- Risk scoring mechanisms for suspicious users
- Automated fraud alert dashboards for administrators
- Continuous AI model retraining using updated datasets

3. Dataset Expansion for AI Models

The system will incorporate larger and more diverse datasets to improve the accuracy of fraud detection and predictive insights. Localized data collection will enhance model reliability within the Pakistani transport context.

4. Predictive Delay Analytics

Future versions may include predictive analytics to forecast route delays based on historical traffic, weather patterns, and booking trends.

5. Mobile Application Development

A dedicated Android and iOS mobile application can improve accessibility, provide push notifications, and increase user adoption.

6. Real-Time Notification Enhancement

Future improvements may include:

- Firebase push notifications
- SMS-based critical alerts
- WebSocket-based live system updates.